A NOVEL ABOUT DATA

# The Data Between Us

*How data really works,*
*for people who think they will never get it*

MEERA & MOHAN · ZAAYKA

*For everyone who was ever handed a spreadsheet*
*and felt a small flutter of doubt.*

*You are more ready for this than you think.*
*Come in, the chai is hot, and the story starts here.*

# Contents

PART ONE

# One Kitchen

*Zaayka is a tiny food-delivery startup in Bengaluru. It is small, and loved, and held together by one tired spreadsheet and the stubbornness of four friends. Before it can grow, Meera and Mohan must lay down the ideas that everything else will one day stand upon. It begins, as the best things do, on an ordinary golden day, over a cup of chai neither of them will ever quite forget.*

· · ·

CHAPTER ONE

# Databases

*The evening everything found its shape*

. . .

It was a slow Sunday afternoon, and then the phone rang. Meera was sitting on the floor of her small flat in Malleshwaram. Outside, a radio played old cricket commentary. The afternoon smelled of coffee and warm dust. It was a good day to do nothing.

The phone was Dev. He was one of her three partners at Zaayka, and a careful man. He double-checked which pocket his keys were in. He worried, so the others did not have to.

"Quick question," Dev said. "How did Sri's shop do on Tuesday? Just that one day. How many orders, and how much money?"

"Give me ten minutes," Meera said.

It was not ten minutes. It was the rest of the afternoon.

To understand why, you need to know one thing about Zaayka. It was a food company, two months old, and it let the city order from small kitchens. From Sri, who made idlis. From Lakshmi, who made biryani. From good little places the big apps ignored.

And every order Zaayka had ever taken lived in one place. A single, giant spreadsheet.

Meera opened it. Four hundred orders looked back at her. She had typed most of them herself, late at night, one by one. She was proud of it. It was the whole life of the company, in one long list.

Now she just had to find Sri's orders from Tuesday, and count them. Simple.

It was not simple.

Meera started scrolling. And straight away, something was wrong.

Sri's shop was not written one way. It was written five ways.

In some rows it said *Sri Idli*. In others, *Sri Tiffin*. In others, just *Sri*. There was a *Sri's*, and even a *SriTiffin* with no space. She had typed them all herself, on different tired nights, never thinking about it. But to the spreadsheet, these were five different shops.

So when Meera counted the rows that said *Sri Idli*, she missed all the rows that said *Sri*. When she counted those, she missed the others. Every count came out different.

She counted once, and got one number. She counted again, and got another. She counted a third time, slowly, and got a third. Three counts, three answers, and no way to know which was true.

| Time | Kitchen | Dish | Price |
|---|---|---|---|
| Tue 12:40 | Sri Idli | Idli plate | ₹90 |
| Tue 13:05 | sri idli | Ghee pongal | ₹110 |
| Tue 13:20 | Sri Idlis | Medu vada | ₹80 |
| Tue 19:15 | SI | Idli plate | ₹90 |
| Tue 21:50 | Sri Idly | Rava dosa | ₹120 |
| Tue 22:10 | Rava dosa | (blank) | ₹120 |

*A few rows from Meera's spreadsheet. To her eyes, one beloved idli shop. To the computer, five complete strangers. And on the last row, the name of a dosa sitting where a kitchen should be.*

She stared at the mess on the screen. A simple question, and she could not answer it.

Meera put the phone face down and stared at the screen. It was nearly dark now. A simple question had eaten her whole afternoon, and she still had no answer for Dev.

That was when the knock came. Two soft raps, then one.

· · ·

It was Mohan.

Mohan had been her friend for six years, since college. He worked with data at a big company across the city. He had a way of looking at a hard thing and finding it easy. It was the most annoying thing about him. It was also, often, the most useful.

He let himself in, because he always did. He looked at her on the floor. He looked at the crossed-out numbers on her notepad. And he smiled.

"You're losing to the spreadsheet," he said.

"It started it," said Meera. "Dev wanted to know how Sri did on Tuesday. That was hours ago. I've counted three times and got three answers."

Mohan sat down on the floor beside her. He looked at the screen for a moment. Then he nodded slowly, like a doctor who has seen this illness many times before.

"Ah," he said. "Sri has five names."

"Sri has five names," Meera agreed, and dropped her face into her hands.

"Can I show you something?" Mohan said. "Not fix it. Just show you why it happened. Then the fix is easy."

"Please."

He picked up her good pen. "Your kitchen. You have a junk drawer, yes? The messy one. What's in it?"

Meera blinked at the change of subject. "Everything. Rubber bands. Old bills. Batteries. A key to a lock we threw away."

"And if I ask you to find the good scissors in there. How long?"

"Ages. You have to empty the whole drawer."

"That drawer," said Mohan, "is your spreadsheet." He let that sit for a second. "Everything is thrown in together. To find one thing, you dig through all of it. That's fine when the drawer is small. But yours has four hundred things in it now, and it grows every day."

Meera looked at the screen. Then at her actual junk drawer, across the room. It was a fair comparison. It was an annoyingly fair comparison.

"So what do I do instead?" she asked.

"You give each thing its own place," Mohan said. "You stop using one messy drawer. You use a set of neat, labelled boxes instead. And a set of neat, labelled boxes has a name. It's called a database."

He said the word gently. Not like a scary computer word, but like a small, useful gift.

"A database is just a very tidy cupboard," he went on. "That's all. But it's tidy in a strict way, and the strictness is the whole point." He explained it slowly, one shelf at a time.

"Picture one shelf, just for kitchens. Every kitchen goes on it exactly once. Sri goes there one time, spelled one way, forever. On that shelf, there is only one Sri. There can never be five."

"One Sri," Meera said softly.

"One Sri. Now picture a second shelf, just for orders. Every order points at the one true Sri on the kitchen shelf. It does not spell his name. It just points."

"Points how?" she asked.

"With a number. On the kitchen shelf, Sri is kitchen number seventeen. So an order just says *kitchen seventeen*. Not his name. Just his number." Mohan smiled. "And you cannot misspell a number. That is the whole trick. The thing that beat you today, gone."

Meera sat with it for a moment, and she saw it. Kitchens on one shelf, each written once. Orders on another, each pointing home by number. To find Sri's Tuesday, there would be no digging, no counting, no five spellings. Just one clean answer.

"That's what I've been missing," she said. "All day."

"Most people miss it their whole lives," said Mohan. "They fight the drawer forever, and blame themselves when they lose."

"There's a bigger idea under all this," he said. "Do you want it?"

"Tell me."

He put the pen down. "Having data is not the same as being able to use it. Look at you. You had every order. You had everything. And you still could not answer one small question."

"Because it was a mess," Meera said.

"Because it had no shape. So here is the real work. You give your data a shape first, on purpose, before anyone asks anything. You do the tidying early. Then, when the question comes, the answer is easy."

He looked at her. "The work is in the organising," he said. "Done before the asking."

Meera liked that. It was a calm idea: do the quiet work first, and the hard thing becomes easy later.

"Say it once more," she said.

"The work is in the organising. Done before the asking."

She wrote it at the top of a clean page. She did not know yet why she wanted to keep it. She just did.

Outside, the light had turned from gold to amber. Down the lane, a temple bell rang three slow notes, spaced apart.

Mohan stood up. He rinsed his chai glass at the sink, without being asked, the way he always did, then stopped at the door.

"Tomorrow," he said. "We build your first proper shelf. One evening. And you will never fight that drawer again."

"Tomorrow," Meera said.

"Bring the good pen. And make your terrible chai. I've grown fond of it." He smiled, and then he was gone. Two soft steps and a wave, down the stairs.

· · ·

Meera sat a while in the quiet flat. Then she picked up the phone and messaged Dev. *Answer tomorrow. A proper one this time.*

She looked at the sentence at the top of her page. *The work is in the organising. Done before the asking.* Then she left the good pen out on the desk, ready for the morning. On the windowsill, the little tulsi plant sat in the last of the light.

Meera slept well that night. For the first time in weeks, she was not dreading a drawer. She had one to tidy in the morning, and now she knew how.

WHAT MEERA LEARNED

She didn't learn a tool today. She learned the *problem* that every tool is built to solve: **having data and being able to use data are two different things.** Meera had every order she'd ever taken, and still couldn't answer one small question, because her data had no agreed shape. Same shop, five spellings. A dish sitting where a kitchen should be.

A **database** is not simply a place to put data. It is a place to keep data in such an agreed, organised way that you can ask it questions and *trust* the answers. Organise first, so the answers come easily later. Every idea in the rest of this book grows from that one small shift: from *where do I store this?* to *how do I store this so it can answer me later?*

*A database is a place that can answer questions.*
*The work is in the organising, done before the asking.*

CHAPTER TWO

# Data Modelling

*What is an order, really?*

. . .

Mohan came back the next evening, as promised, and Meera had the good pen ready.

She had thought about the drawer all day. The idea of neat, labelled boxes had stayed with her. She wanted to build one. But when she sat down to start, she found she did not know where to begin.

That was the first thing she told him when he knocked.

"I tried to start," she said. "And I got stuck at the very first step. You said give things their own boxes. But I don't even know what my boxes are. What is a box, here? What am I actually storing?"

Mohan smiled and sat down on the floor. He looked pleased, not worried.

"That is the right place to be stuck," he said. "Most people never even ask that question. They just start typing and hope. You stopped and asked what you're really storing. That question has a name. It's called data modelling. And it is the most important thing we will do."

"Let's start with one thing," Mohan said. "The most ordinary thing Zaayka has. An order. Somebody wants food. Tell me what an order actually is."

Meera thought. "Someone orders food. It comes from a kitchen. They pay. It arrives."

"Good. But that's a story, not a shape. It's vague. A computer can't hold *someone wants food*, *somehow*, *sometime*." He tapped the notepad. "So we do a thing called modelling. We take that vague idea and turn it into a small, clear set of facts. Named facts. Each one a single, definite thing."

"What do you mean by a fact?"

"Let me show you. We'll pull the real facts out of your story, one at a time."

He wrote as he spoke.

"Who placed the order? That's one fact. Which kitchen? Another. When did it happen? Another. How much did it come to? Another." He looked up. "And one more, the most important. Each order needs its own name. A number that belongs only to it. So we can always tell one order from another."

Meera watched the list grow. It was turning from a vague idea into something with edges.

"So an order becomes five clear facts," she said. "Its own number. The customer. The kitchen. The time. The total."

"Exactly. And each of those facts has a name. We call a single named fact a **field**." He drew it out properly now, a small table with a name at the top and the fields as columns.

**ORDER** · first draft, deliberately simple

| order_id | customer | kitchen | placed_at | total |
|---|---|---|---|---|
| 4402 | Priya | Sri Idli | 2026-03-10 20:04 | ₹370 |

**order_id** *is a* ***key****, a unique handle for this order. We'll return to keys properly soon.*

"There," said Mohan. "That is one order, modelled. Order number 4402. Placed by Priya. From Sri's shop. At four minutes past

eight, on the tenth of March. Three hundred and seventy rupees in total. Five fields. Each one clear. Each one a single fact."

Meera looked at it for a while. It was a small thing. It was also, she realised, exactly what her spreadsheet had never had. Clear, named facts, agreed in advance.

"Now, one careful point," Mohan said. "Look at the last field. The total. Three hundred and seventy rupees."

"What about it?"

"That is the total for the *whole* order. But Priya didn't order one thing for three hundred and seventy rupees. She ordered a few dishes, and they added up to that. The dishes each have their own price. The order has a total. Those are not the same fact." He looked at her. "I'm pointing at it now, gently, because one day that difference will matter a great deal."

Meera nodded slowly. Then she frowned at the little table.

"Wait," she said. "Then where are the dishes? This says Priya spent three hundred and seventy rupees at Sri's shop. But it doesn't say what she actually ate. The dishes aren't here at all."

Mohan's smile widened. He looked like a man whose favourite part had just arrived.

"You found it," he said. "You found the exact place this simple version breaks. And you found it yourself." He held up a hand. "I'm not going to fix it tonight. I want you to feel the gap first. This little table is the honest, too-simple version. One order, as one flat row. It's good enough to show you what a field is. It is nowhere near good enough to be true."

"So what makes it true?"

"The dishes. Tomorrow. It turns out to be the most important idea in the whole book, and you just walked right up to its door." He tapped the word *field* on the page. "But first, one more thing about fields, because it's quietly powerful."

"Every field has a name," he said, "and it also has a *type*. A type is the kind of thing the field is allowed to hold."

He pointed down the little table, field by field.

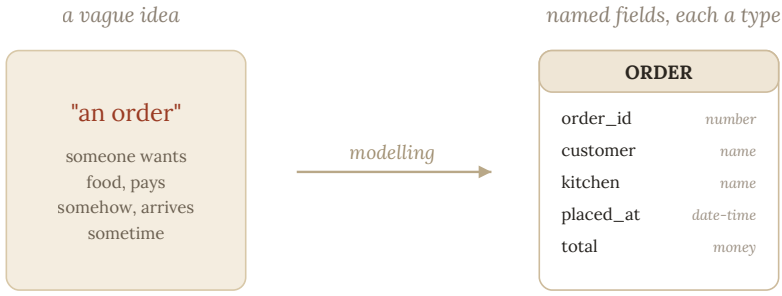
"The order number is a whole number. The customer is a name, made of letters. The time is a date and a clock time, nothing else. The total is money." He looked at her. "Each field promises what kind of value lives in it. And it keeps that promise, in every single row, forever."

"Why does that matter so much?"

"Because the promise protects you. If the time field must hold a real date, then nobody can type *Tuesdayish* into it. Not even you, at midnight, half asleep." He smiled. "Half the mess in your spreadsheet is there because a spreadsheet cell will swallow anything. A name, a number, a little poem. A proper field will not. It has standards."

Meera laughed. She liked that. A field with standards. It seemed a very Mohan sort of thing for a field to have.

He showed her the whole idea drawn out. The vague story on one side. The clear, named, typed fields on the other. The turning of one into the other.




*Data modelling turns a vague idea into a clear shape: named fields, each promising one type of fact. This first draft is deliberately simple; the next chapter makes it true.*

"That," said Mohan, "is data modelling. You take a fog, and you turn it into a shape. Named facts. Fixed types. Agreed before anyone asks a question." He sat back. "It's the same thing you learned yesterday, really. The work is in the organising. This is just the organising, done carefully, one field at a time."

· · ·

He left soon after, with the usual wave and the rinsed glass and the two soft steps down the stairs. But Meera stayed up a while.

She looked at the little order she had helped build. Order 4402, standing there with its five clear fields, patient and true. It was a small thing. But it was hers, and it was correct, and nothing about it could be spelled five ways.

She found she did not want to stop yet. So she drew a second box beside the first, an empty one, and above it she wrote a single

word. Then she sat and looked at the two boxes together. The full one, and the empty one. Clearly meant to be joined, and not yet joined.

The word above the empty box was DISHES.

### WHAT MEERA LEARNED

**Data modelling** is deciding the shape of your world *before* you store it. You take a vague idea like "an order" and point at the real, separate things hiding inside it. Each real thing your business cares about, an order, a kitchen, a customer, a dish, is an **entity**, and it deserves its own shape and home. Squeezing one whole entity into a single column of another (a kitchen crammed into an order) is exactly what causes the mess.

Each shape is built from **fields**: single named facts, each with an agreed **type**. A type is a promise the computer can enforce, so a date field refuses a mood and a money field refuses a word. The shape defends itself.

One honest note carried forward: an order has a *total*, while the dishes within it have their own *prices*. Those are different facts, and keeping them straight will matter very soon.

*Name the things that matter, give each its own shape, and let every fact declare what kind of fact it is.*

CHAPTER THREE

# One Order, Many Items

*The dish that would not fit in a box*

. . .

The empty box marked DISHES was still there in the morning, waiting for her.

Meera had slept well, but the box had nagged at her, in a good way. She wanted to fill it. But she could not quite see how the dishes fit. So she did a thing that would have surprised her a month ago. She went to see Sri.

Sri ran the idli shop that had started all this trouble, kitchen number seventeen. He had been there thirty years. It had been his father's shop before him. If anyone understood what an order really was, it was him.

. . .

Sri's shop at ten in the morning was busy and warm. Steam rose off the idli pot. The griddle hissed. Sri stood at the counter, calling orders back to the kitchen without writing a single one down. Thirty years had made writing them down unnecessary.

Meera bought a coffee she did not need and stood to one side, and watched.

A man asked for two plates of idli and a vada. A student took one masala dosa to go. A grandmother ordered three different

tiffins for three different grandchildren, and paid from coins knotted in a handkerchief.

And standing there, coffee cooling in her hand, Meera saw it.

Every customer was one order. But not one of them wanted only one thing. Two idlis and a vada. Three tiffins. Every order, the moment it became real, opened up into a little list of dishes. The thing she had tried to squeeze into a single box last night simply refused to be one line. It was always one order, with several dishes inside it.

"One order, many dishes," she said out loud, to nobody. A man waiting for his parcel gave her a look.

And now Sunday made sense to her, in a new way.

Her old spreadsheet had held one dish to a line. So a customer who ordered three things had quietly become three separate rows. And on each tired row, she had spelled Sri's shop a little differently. She had been tearing whole orders into pieces every night, without ever seeing her own hands do it.

She did not care that the parcel man was still watching. She had worked it out herself, standing in a shop, before anyone told her the answer. She pulled out her phone and called Mohan.

He answered on the second ring. She could hear his office behind him.

"I'm at Sri's," she said, with no hello. "I worked out the dishes. An order isn't one dish in a box. It's one order, with a whole list of dishes hanging off it. Two dosas, an idli, a coffee, all under the same order. I've watched it happen twenty times this morning."

There was a short pause on the line. When Mohan spoke, the teasing had gone out of his voice.

"Say that again," he said. "The shape of it. What did you just see?"

"One order has many dishes. One on this side. Many on the other."

"Meera. That little phrase, *one has many*, is the shape at the centre of almost every system you will ever build. It has a name. It's called a **one-to-many** relationship. And you didn't read it in a book. You stood in a shop and watched it happen." He paused. "Stay there. Order a real breakfast, and keep me on the line. I'll talk you through where the dishes were always meant to live."

· · ·

So she stayed, phone warm against her ear, coffee cooling at Sri's counter, and Mohan talked her through it from his desk across the city.

"When one thing owns several of another," he said, "you don't cram the many into the one. You give the many their own place. A second table, beside the first."

"So the dishes leave the order," Meera said, "and go live next door?"

"Next door, but never strangers. The order keeps what's true about the whole order. Who placed it, which kitchen, when, and the total. And the dishes move out into a table of their own, one row for each dish." He paused. "One small thing, though. You wrote DISHES last night. Good word. But look at what a single line really needs to remember. Not just *masala dosa*. Also how many. Also the price of each. Also what that line came to."

"So it's more than a dish," Meera said. "It's a whole line off the bill."

"It's an *order item*. That's its honest name. So that's what we'll call the table. Same idea you had last night, just grown up a little overnight."

Meera crossed out DISHES in her notebook and wrote ORDER_ITEMS above it. Then Mohan showed her Priya's order, split the way it had always wanted to be split. First, the order itself, holding only what was true about the whole thing.

**ORDERS**

| order_id | customer_id | kitchen_id | placed_at | total |
|---|---|---|---|---|
| 4402 | 318 | 17 | 2026-03-10 20:04 | ₹370 |

*One row: what is true about the whole order. Notice it keeps the total, ₹370, but not the dishes.*

Then the dishes, in their own table, one row each.

**ORDER_ITEMS**

| item_id | order_id | dish | qty | unit_price | line_total |
|---|---|---|---|---|---|
| 9001 | 4402 | Masala dosa | 2 | ₹120 | ₹240 |
| 9002 | 4402 | Idli plate | 1 | ₹90 | ₹90 |
| 9003 | 4402 | Filter coffee | 1 | ₹40 | ₹40 |

*Three rows, one per dish. Each carries* **order_id** *= 4402 so it knows its order. (That pointer is next chapter's subject.)*

Meera studied the two tables at Sri's counter, and the whole thing clicked into place.

"There it is," she said. "The order keeps the total. Three hundred and seventy. And each dish keeps its own price, over here. Two dosas at a hundred and twenty is two hundred and forty. Add the idli and the coffee, and you're back to three seventy." She looked up. "This is exactly the thing you flagged last night. The total, and the prices. They live in different tables now, and I finally see why."

"That," said Mohan, and the warmth came back into his voice, "is exactly what I flagged. And look what you can do now. You've kept the total *and* the pieces. So you can check one against the other. If the dishes don't add up to the order total, you know at once that something is wrong." He paused. "A good shape doesn't only hold your facts. It lets your facts keep each other honest."

Sri chose that moment to slide a plate across the counter to her, unasked. One masala dosa, crisp and gold. A small steel tumbler of chutney beside it. He had not taken her order. He had simply known.

"On the house," Sri said, already turning to his next customer. "You've been standing in my shop staring at your phone like it owes you money. Eat something, beta."

Meera looked at the plate. One order. Many items. A dosa and a chutney, given freely by a man who had carried the shape of all this in his bones for thirty years, without ever needing a word for it.

She put the phone away and ate. It was, she thought, the best dosa in three neighbourhoods. And now, at last, that fact could be stored, and proven, and trusted.

WHAT MEERA LEARNED

When one thing naturally owns several of another, that is a **one-to-many** relationship, the most common shape in all of data. One order has many items. One kitchen has many menu dishes. One customer has many orders.

The rule is exact: when you see *one has many*, you have two entities, not one, and the "many" side gets its own table. Each row there remembers which "one" it belongs to. Cramming the many into a single cell hides them from every question; smearing them across copied rows invites the same errors as copying a kitchen's name. Giving them their own home fixes both.

It also puts each fact where it belongs: the order keeps its *total*, while each item keeps its own *unit price* and *line total*, so the pieces can be added up and checked against the whole.

*When one thing has many of another,*
*give the many a table, and let each remember its one.*

CHAPTER FOUR

# How Things Hold Hands

*Keys, and the end of five spellings*

. . .

Meera went back to Sri's shop that evening, and this time she brought Mohan with her.

She had wanted to show Sri what she had built. It was his five spellings, after all, that had started everything. So when the lunch rush was over and the shop was quiet, the three of them sat with a notebook between them.

Mohan arrived hungry, as always. He greeted Sri like an old friend and stole a hot vada off the counter before anyone could stop him. He paid for it with a grin instead of coins. Sri pretended to be furious. He was not furious. Nobody stayed furious with Mohan for long.

Meera showed Sri the notebook. The orders. The dishes in their own table. Sri wiped his hands on his apron and looked at it for a long moment. Then he asked a question that stopped her cold.

"So in this machine of yours," Sri said slowly, "what am I?"

Meera opened her mouth, and found nothing in it.

"I mean it," Sri went on. "Thirty years I have this shop. My father's before me. Now you tell me I am somewhere inside your computer. So what am I in there? A name? I have seen how the name goes. Five ways you spelled me. You told me yourself, laughing." He

was not laughing. "If I am only a name, and the name can be spelled five ways, then which of the five is me?"

The little shop went quiet.

Meera looked at Mohan, a little helpless. But Mohan did not step in with the answer. He only tilted his head at her, the way he did when he had handed her something on purpose.

"That," Mohan said gently, at last, "is the best question anyone has asked me all year. And Meera is going to answer it. Because she already knows. She just doesn't know she knows yet."

"I do?"

"When we first drew an order, you gave it a number. Its own number. Why a number? Why not just its details?"

Meera thought. "Because two orders could look the same. Same customer, same kitchen, same evening. The number is the one thing that is only theirs. It's how you tell them apart, for certain."

"So the number is the thing that can never be confused with anything else. The one true handle on that order." Mohan turned to Sri, and his voice softened. "Sri. Your name can be spelled five ways because a name is a description, and descriptions are slippery. You don't need a better spelling. You need a number of your own. One that is only yours. One that no tired thumb at midnight can ever misspell, because there is nothing to spell."

He wrote a small table on a fresh page. A table just for kitchens.

**KITCHENS**

| kitchen_id | name | area |
|---|---|---|
| 17 | Sri Idli | Malleshwaram |
| 18 | Krishna Bakery | Malleshwaram |
| 19 | Lakshmi's Biryani | Rajajinagar |

*The name lives in exactly one place. One row. One spelling.*

"There," Mohan said. "You are not a name in the computer, Sri. You are **kitchen number seventeen**. Only you. Forever. Sri Idli, in Malleshwaram, one row, one number. Every order that ever comes to you will point at that number, not at a spelling. You can misspell the name on a screen all you like. The number underneath does not move."

Sri looked at the little table. At the 17 beside his shop's name. Something in his face eased, the way a man eases when a worry he could never quite say out loud is finally answered.

"Seventeen," he repeated. "I am seventeen."

"You are seventeen. And seventeen is the opposite of being forgotten." Mohan said it simply. "A name can blur. A number holds. It has a proper title, too. It's called a **primary key**. The one field that is unique for every row, never empty, never changing. The true name of a thing, in a world that cannot spell."

Meera wrote PRIMARY KEY in her notebook. And beside it, without quite deciding to, she wrote: *the opposite of being forgotten.*

"Now watch what the key lets you do," Mohan said, warming to it. "Your orders were carrying the kitchen's *name*. That was the whole disaster. So let's stop. Let each order carry the kitchen's *number* instead."

He laid the orders table beside the kitchens table.

**ORDERS** · now with a kitchen_id

| order_id | customer_id | kitchen_id | placed_at | total |
|---|---|---|---|---|
| 4402 | 318 | 17 | 2026-03-10 20:04 | ₹370 |
| 4419 | 206 | 17 | 2026-03-10 13:12 | ₹150 |
| 4404 | 318 | 17 | 2026-03-10 21:55 | ₹210 |

**kitchen_id** *points at kitchens. Every order from Sri Idli just carries* **17**.

"Look at that," he said. "The order no longer says *Sri Idli*, spelled however the night felt. It says kitchen seventeen. It points at the one true row in the kitchens table, where the name lives exactly once."

Meera felt the whole thing lock into place.

"So the name only lives in one place now," she said. "In the kitchens table. Everything else just points to it by number. If Sri ever repaints his sign, I change that one row, and every order in history shows the new name at once. Because they were never holding the name at all. Only the number."

Mohan looked at her a second longer than the moment needed.

"You just described why the whole world builds databases this way," he said. "One fact, one home. Everyone else points. That pointer, the number in one table that refers to a key in another, is called a **foreign key**." He smiled. "Primary key: my own true name. Foreign key: my finger, pointing at yours."

"Primary key at home," Meera said. "Foreign key pointing away."

"And with just those two ideas, you can wire a whole company together. Orders point to kitchens. Dishes point to orders." He nodded at the order-items table. "Look. Each dish carries the order's number, 4402. A foreign key, pointing back at the order it belongs to."

ORDER_ITEMS

| item_id | order_id | dish | qty | unit_price | line_total |
|---|---|---|---|---|---|
| 9001 | 4402 | Masala dosa | 2 | ₹120 | ₹240 |
| 9002 | 4402 | Idli plate | 1 | ₹90 | ₹90 |
| 9003 | 4402 | Filter coffee | 1 | ₹40 | ₹40 |
| 9004 | 4419 | Ghee pongal | 1 | ₹110 | ₹110 |

**item_id** *is this table's primary key.* **order_id** *is a* **foreign key***: it points back to a row in orders.*

"That was a foreign key all along," Meera said. "This morning, the dishes holding the hand of their order. I just didn't have the word for it yet."

"You had the idea. The word is only a coat we put on it." He turned back to the orders. "And here is the orders table itself, with its own true name shining at the front."

ORDERS

| order_id | customer_id | kitchen_id | placed_at | total |
|---|---|---|---|---|
| 4402 | 318 | 17 | 2026-03-10 20:04 | ₹370 |
| 4419 | 206 | 17 | 2026-03-10 13:12 | ₹150 |
| 4404 | 318 | 17 | 2026-03-10 21:55 | ₹210 |

**order_id** *is the primary key: one unique number per order.*

"One last thing," Mohan said, "and this one is my favourite." Sri, who had been listening all the while as he wiped his counter, leaned in despite himself.

"Some connections don't run just one way," Mohan said. "Take favourites. A customer can favourite many kitchens. And a kitchen

can be favourited by many customers. Priya loves Sri's idlis and Lakshmi's biryani both. And Sri here is loved by half of Malleshwaram." Sri did not deny it. "So it isn't one-to-many. It's many on both sides at once. And you cannot fit that into either table without making a mess."

"So where does it go?" Meera asked.

"You give the relationship its own little table. A table whose only job is to hold hands. Each row is one connection. One customer, one kitchen. They like each other. Nothing else."

**FAVOURITES**

| customer_id | kitchen_id |
|---|---|
| 318 | 17 |
| 318 | 19 |
| 206 | 17 |

*Each row is one connection. Customer 318 favourites kitchens 17 and 19. Customer 206 favourites 17. This is a **join table**.*

"A table just for holding hands," Meera said, and laughed. "That's the sweetest thing you've taught me."

"It has a very unromantic name, I'm afraid. A **join table**." His eyes were laughing at her. "But you had the better word for it. In this whole cold machine of keys and pointers, the thing that lets two entities belong to each other, freely, in every direction, is a little table that exists only to hold them together. There are worse ways to think about it."

Meera met his look, and then, for reasons she did not stop to examine, looked back down at her notebook rather quickly.

· · ·

Sri walked them to the door as the shutter came halfway down. At the threshold he stopped Meera. He was a little moved, in the gruff way of a man who does not like to be caught at it.

"Seventeen," he said again. "For thirty years I have worried this shop would be forgotten one day. My father's name, gone off the street." He tapped the notebook in her hand. "Now you tell me there is a number that is only mine. That cannot be spelled wrong. That every order will point to, forever." He nodded once. "Look after my seventeen, beta."

"I will," Meera said, and meant it more than she had expected to. "I promise."

They stepped out into the evening. Behind them, Sri's shutter rattled the rest of the way down, over a shop that was, for the first time in thirty years, impossible to misspell.

Mohan glanced sideways at her as they walked. "You gave a man back thirty years of worry tonight, with a two-digit number. Not bad for someone who couldn't count her own orders on Sunday."

"I had a good teacher," she said.

"You had a *very* good teacher," he agreed, insufferable, and stole the last bite of the vada he had never actually paid for.

Meera laughed into the warm dark. And she did not think, yet, about why the evening felt like something she wanted to keep.

WHAT MEERA LEARNED

Every important thing gets a **primary key**: one permanent, unique value of its own, usually a number, that tells it apart from everything else with total certainty. ORDER_ID 4402, KITCHEN_ID 17. Names repeat; keys never do.

When one row needs to point at another, it carries that other row's key. A key carried to point elsewhere is a **foreign key**. This lets you keep every fact in exactly one place, an idea called **normalisation**: the kitchen's name lives in one row of KITCHENS, and every order simply carries KITCHEN_ID. You cannot misspell a number, so whole classes of mess vanish. Change the fact once, and everyone pointing at it is instantly correct.

A **one-to-many** relationship (one order, many items) is handled by putting the foreign key on the "many" side. A **many-to-many** relationship (customers and their favourite kitchens) is handled by a **join table** that holds one row per connection.

*Keep every fact in one place, and give it a key.*
*Then let everything else simply point.*

CHAPTER FIVE

# What Kind of Fact Is This?

*Rupees, dates, and the danger of "nearly"*

· · ·

Lakshmi was angry, and she wanted Meera to know it.

Meera had met Sri already. Lakshmi she had only heard about. Lakshmi ran the biryani kitchen two neighbourhoods over, kitchen number nineteen. She had cooked there for thirty years. She ran it like a captain runs a ship, and she was proud of one thing above all: she had never once touched a computer.

So when Lakshmi called the office, upset, Meera went to see her in person.

· · ·

The biryani kitchen smelled of saffron and slow-cooked rice. Lakshmi stood behind her counter with a ledger, a real paper one, and she tapped it with one finger.

"Your machine says I earned two rupees less than I did," she said. "Every day. Two rupees. Small money. But it is my money, and it is wrong. Thirty years I count by hand and I am never wrong. Your machine is wrong. Explain it to me."

Meera looked at the numbers. Lakshmi was right. The app's total for the day was two rupees short of Lakshmi's hand-count. Not a lot. But wrong is wrong.

She did not know why. So she told Lakshmi the truth. "I don't know yet. But I will find out, and I will come back and show you. I promise."

Lakshmi studied her for a moment. Then she nodded, once, and went back to her rice.

· · ·

That evening, Meera laid the problem in front of Mohan.

"Two rupees short, every single day," she said. "It's so small. But Lakshmi counts by hand and she's never wrong, and she trusts her hand more than my whole app. Which, honestly, so do I right now."

Mohan did not look worried. He looked interested. "Show me how the app adds up her prices."

Meera showed him. And Mohan smiled a small, knowing smile.

"Ah," he said. "This is a famous trap. It catches everyone once. Let me show you the strangest thing a computer does." He opened her laptop and typed something very simple. Just three small numbers, added up. Ten paise, plus ten paise, plus ten paise."

"That's thirty paise," Meera said. "Obviously."

"Watch what the computer says."

```
# asking the computer, the careless way
0.1 + 0.1 + 0.1  # three ten-paisa pieces
# the computer answers:
0.30000000000000004
```

Meera stared at the screen. Where she expected a clean 0.30, there was a long ugly tail of digits. Zero point three, and then a string of zeros, and then, at the very end, a stray four.

"What *is* that?" she said.

"To understand it," Mohan said, "you need one idea we've touched but never opened up. Types."

"You mentioned types. When we built the order. Each field has a type."

"Right. A **type** is the kind of value a field is allowed to hold. And there is more than one kind of number." He wrote as he spoke. "There is the whole number. One, two, seventeen. No fractions. It's exact, and it's perfect for counting things. Orders, quantities, the number seventeen."

"And for money? Money has paise. Fractions."

"That's the trap. There are two ways a computer can hold a fractional number, and only one of them is safe for money." He drew the two side by side. "One way is called a *floating-point* number. It's fast, and it's clever, and it's how the computer did that sum just now. But it does not store most fractions exactly. It stores something extremely close. Close enough for science. Not close enough for money."

"So the tiny error I saw..."

"Is the floating-point number being *almost* right. One paisa, a hundred times a day, rounding a hair the wrong way. And it piles up. Two rupees a day, in Lakshmi's case." He looked at her. "She was right. Her hand was right. Your computer was doing arithmetic that is fine for a physicist and wrong for a shopkeeper."

Meera felt something settle. Lakshmi had been right all along, and now she knew why.

"So what's the safe way?" she asked. "How do I hold money without the error?"

"You use the other kind of fractional number. It's called a *decimal* type. It stores the value exactly, to the paise, no stray tails. It's a little slower. Nobody cares. You use it for every rupee in the

company." He wrote out the common types, plainly, one line each, and the trap that comes with each.

### THE EVERYDAY TYPES

| type | holds | good for | a trap to avoid |
|---|---|---|---|
| **integer** | whole numbers | ids, counts, quantity | for money, store exact paise, never loose rupees |
| **decimal** | exact fractions | money, precise amounts | the one to use for rupees |
| **text** | words, any length | names, notes, addresses | don't store numbers you'll do maths on |
| **boolean** | true or false | is_paid, is_open | keep it truly two-valued |
| **timestamp** | a date and time | placed_at, delivered_at | mind the time zone (next) |
| **date** | a calendar day | birthdays, report days | a day is not a moment |

*Six types cover almost everything you will ever store. The art is choosing the right one on purpose.*

"Read it like a shopping list," Mohan said. "Whole numbers for counting. Decimal for money, always. Text for words. A proper date-and-time for moments. Each type is a promise about what lives in the field, and how it behaves." He tapped the money row. "And the one rule you must never break: never hold money in a floating-point number. Use exact decimal, or count in whole paise. Always."

Meera wrote it down and underlined it twice. It was the first rule that had come from a real person losing real money.

"There's one more thing while we're here," Mohan said. "It's small, and it trips people up constantly. Look at your orders again."

He pulled up the orders and pointed at one that had been placed but not yet delivered.

**ORDERS** · some deliveries haven't happened yet

| order_id | kitchen_id | placed_at | delivered_at | total |
|---|---|---|---|---|
| 4402 | 17 | 20:04 | 20:38 | ₹370 |
| 4419 | 17 | 13:12 | 13:44 | ₹150 |
| 4404 | 17 | 21:55 | NULL | ₹210 |

*NULL is not zero and not blank. It means "no value yet." Order 4404 is still cooking, so it honestly has no delivery time.*

"See the delivery time on that last order? It's empty. The order was placed, but the food hasn't arrived yet. So what do we put in the *delivered* field?"

"Nothing? Zero?"

"Not zero. Zero is a time, midnight. And not blank, because blank is sloppy and means nothing clear." He wrote a single word. "There is a special value for exactly this. It's called **NULL**. And NULL means one precise thing: *no value yet.* Not zero. Not empty. Just, honestly, *we do not know this yet.*"

"So NULL is the computer being honest that something hasn't happened."

"That's it exactly. The delivery time is NULL until the food arrives. Then it becomes a real time. NULL is how a database says *not yet* without pretending to know something it doesn't." He smiled. "It's a small, honest word. You'll come to like it."

"And the last idea," Mohan said, "is the one that would have caught Lakshmi's bug before it ever reached her. A **constraint**."

"A constraint?"

"A rule the database itself refuses to break. You can tell a field: you must never be empty. Or: you must always be a positive number. Or: this must be money, held exactly. Once you set the rule,

the database guards it, in every row, forever, without ever getting tired." He looked at her. "A type is a promise. A constraint is a promise the database *enforces*. It stands at the door and turns away anything that breaks the rule. Even you, at midnight."

Meera thought about all the mess in her old spreadsheet. Every bit of it had got in because a spreadsheet cell would accept anything at all.

"So a constraint is the opposite of my old spreadsheet," she said. "The spreadsheet swallowed anything. A constraint refuses the wrong thing at the door."

"The opposite of your old spreadsheet." He grinned. "That's a good way to put it. Types and constraints, together, are how a database keeps its own promises. They're why you can trust the answer at the end."

· · ·

The next morning, Meera went back to Lakshmi's kitchen with the fix in hand.

She had changed every price in the company to the exact decimal type. The two-rupee error was gone. She showed Lakshmi the new total, and it matched the paper ledger, to the last paisa.

Lakshmi looked at the two numbers, side by side, the machine's and her own. They agreed. She looked at Meera for a long moment.

"Thirty years I trust my hand," Lakshmi said at last. "Not a machine." She tapped the matching total, once. "But maybe I trust the girl who fixes the machine when it is wrong. That is a different thing."

It was, Meera thought walking home, the best thing anyone had said to her in weeks. Not that the machine was trusted. That she was.

She told Mohan later, and he went still in the way he did when something pleased him. She had started to know that stillness by now. She had begun, without meaning to, to learn his face.

WHAT MEERA LEARNED

Every field carries a **type**, a promise about what kind of value lives there and how it behaves. Choosing types on purpose is real engineering, and one choice matters above all others: **never store money in a floating-point number**, because floats carry tiny errors that pile up. Use an exact decimal type for money (or whole paise), always.

Store every moment of time in one universal clock (**UTC**) and convert to local time only when showing it to a person, so machines in different cities never disagree about when things happened.

When a fact is genuinely not known yet, store **NULL**, the honest absence of a value. NULL is not zero and not blank. Because it means "unknown," comparing anything to NULL yields "unknown," which is why it must be handled with care.

Beyond types, **constraints** defend the data: NOT NULL (required), UNIQUE (no duplicates), PRIMARY KEY (unique and not null), and the **foreign key** constraint, which refuses to let a row point at something that does not exist. Types guard the values; constraints guard the relationships.

*A good shape does not only hold your data.*
*It defends it, especially when you are tired.*

CHAPTER SIX

# Asking the First Question

*SELECT, and the end of counting by hand*

. . .

For three weeks, Meera had been building a house she was not yet allowed to live in.

Evening by evening, she and Mohan had built it. Tables with clear fields. Types that kept their promises. Keys, so that Sri was one shop and not five. Constraints, standing guard at every door. She had moved all four hundred orders into it, by hand, one at a time.

But she had not yet asked it a single question. That had been the deal. Build the house first. Then learn to speak to it.

Tonight was the night. She had been thinking about it all day, the way you think about a festival.

. . .

Mohan arrived with an air of occasion, which for Mohan meant he had brought two kinds of biscuits instead of one.

"Tonight," he said, settling onto the floor, "you stop counting with a pencil like a shopkeeper in an old film. Tonight you learn to ask. And the asking has a language."

"A language?"

"It's called **SQL**. People make it sound frightening. It isn't. It was built, long ago, to read almost like plain English. So that ordinary people, asking ordinary questions, would feel at home." He turned the laptop to face her. "Let me show you the easiest question there is. Read it out loud. Tell me what you think it will do."

```
SELECT * FROM kitchens;
```

"Select star from kitchens." Meera frowned, then heard herself. "Select everything from the kitchens table. Just… show me all the kitchens?"

"Press it and see."

She pressed it. And there they were. Her kitchens, all of them, rising onto the screen in a neat and obedient list.

KITCHENS

| kitchen_id | name | area |
|---|---|---|
| 17 | Sri Idli | Malleshwaram |
| 18 | Krishna Bakery | Malleshwaram |
| 19 | Lakshmi's Biryani | Rajajinagar |

Meera stared. Sri was there, one Sri, spelled one way. Lakshmi. Krishna Bakery. The whole little world she had spent three weeks tidying. And it had answered her the instant she asked. No wrong spellings. No counting by hand.

"Oh," she said softly. It was the sound of a door opening.

"Let me name the parts," Mohan said. "SELECT means *what do you want to see*. FROM means *which table to look in*. And the star means *everything, every column*. Useful when you're peeking. But

usually you don't want everything. You want a few things. So you name them instead of the star."

He showed her the next one.

```
SELECT name, area FROM kitchens;
```

"Just the name and the area," Meera said. "I ask for less, I get less. It gives me exactly what I name." She was already reaching for the keyboard. "So SELECT is what I want. FROM is where to look. What if I only want *some* of the kitchens? Only the ones near Sri?"

"Then you add a third word," Mohan said. "WHERE. It's a filter. It keeps only the rows that pass a test."

```
SELECT name, area
FROM kitchens
WHERE area = 'Malleshwaram';
```

"Where area equals Malleshwaram." She ran it, and the list shortened to just the neighbours, Sri and the bakery, the two kitchens on her own lane. She laughed. "It listened. I gave it a rule, and it only kept the rows that obeyed."

"Three words," Mohan said. "SELECT, FROM, WHERE. What to show. Where to look. Which rows to keep. You now hold most of the language. Everything else is decoration on those three."

"All right," he said, and something in his voice made her look up. "You've been carrying a question for three weeks. Since the very first evening. Dev asked it, and it sent you to the floor with a pencil, and it beat you. Do you remember it?"

Of course she remembered it. It had started all of this.

"How did Sri's shop do," she said. "On Tuesday. The tenth."

"Ask it. Not me. Ask the house you built."

Meera's hands went still on the keys for a moment. Then she began to type, slowly, saying it under her breath as she went. Show me the order, when it was placed, and the total. From the orders table. Where the kitchen is seventeen. And the day is the tenth.

```
SELECT order_id, placed_at, total
FROM orders
WHERE kitchen_id = 17
  AND placed_at >= '2026-03-10'
  AND placed_at < '2026-03-11';
```

She did not press it at once. She looked at it, this small paragraph of plain words that was somehow also a question. Then she pressed it.

The answer came back before she had finished breathing in. Every one of Sri's orders for that day. Lined up. Counted. True.

No pencil. No floor. No five spellings. Three weeks ago this exact question had cost her a whole Sunday and given her three wrong answers. Now it cost her one breath and gave her the right one.

Meera did not say anything for a moment. Her eyes had gone bright.

"It just answered me," she said. "I asked, and it just... answered."

"It did."

"The thing that broke me. That I cried a little bit about, that first night, if you must know." She turned to him, her whole face lit. "I just asked it in four lines. And it told me the truth."

And here is the thing she did not plan. Her very first instinct, before Dev, before anyone, was to show *him*. Not to tell Mohan the answer, he already knew it. To let him see her face while she found

it. As though the answer was only half real until he had watched her arrive at it.

Mohan was not looking at the screen at all. He was watching her, the way you watch the first rain after a long dry month. When she caught him at it, he glanced away and reached, unhurried, for a biscuit.

"One more," he said. "You'll like this one. You have Sri's orders for the day. Which was his biggest? Don't count. Just ask the house to sort them for you. Largest first."

```
SELECT order_id, placed_at, total
FROM orders
WHERE kitchen_id = 17
ORDER BY total DESC;
```

"Order by total, descending," Meera read. "Put them in order. Biggest at the top." She ran it, and Sri's fattest order of the day rose to the first row. "It even arranges them for me. I just say how I want them laid out, and it lays them out."

"That," said Mohan, "is the end of counting by hand. Forever. Not just for this question. For every question you will ever have about your own company." He looked at her. "You stopped being someone who *keeps* data tonight, Meera. You became someone who can *ask* it things."

· · ·

Later, when he had gone, Meera did a small thing she told no one about.

She opened the laptop again, alone, and asked the house one more question. Not for Dev. Not for the company. Just for herself. She asked it to show her every order Zaayka had ever taken, all

four hundred, from the very first to the last. And she sat and watched them fill the screen, top to bottom. A whole young company, answering her in a single quiet rush.

Three weeks ago this had been a drawer of chits that beat her on a Sunday. Now it was a house that answered when she spoke. She had built it herself, mostly. And it had learned her voice.

Then she sent Dev the true number, the one he had asked for three weeks ago. It was right, and she knew it was right. And that knowing was a feeling she had never once had in all her months of spreadsheets.

She turned off the lamp. On the windowsill, the tulsi kept its small green watch. And Meera fell asleep happier than a person has any right to be over four lines of text.

WHAT MEERA LEARNED

**SQL** is the language for asking a database questions, written to read almost like plain English. Four words do an enormous amount of work. SELECT chooses which columns to show (a * means all of them). FROM names the table. WHERE filters to just the rows that pass a test, and conditions can be stacked with AND. ORDER BY sorts the result, with DESC for largest first.

Two small but real professional habits appeared: text values go in quotes while numbers do not; and to filter to a single day, ask for a *range* (from the day, up to but not including the next day) rather than an exact instant, because a day is a span of moments, not one point in time.

With these, Meera finally answered the question she could not answer on that first afternoon, in one second, correctly, because the idli shop is no longer five spellings but one clean KITCHEN_ID.

*The first question is a door you walk through once. After it, the data answers to you, not the other way round.*

CHAPTER SEVEN

# Questions That Add Up

*Grouping, counting, and joining tables back together*

. . .

The terrace above Meera's flat was the best-kept secret in the building.

Four floors up, past the water tanks, there was a flat roof with a low wall and a view of the whole neighbourhood going about its evening. Someone had left two plastic chairs up there years ago, and nobody had ever taken them away. On warm nights it caught whatever breeze the city could spare.

Meera had started bringing the laptop up here. Tonight she had brought Mohan too.

"Dev asked me a new question," she said, settling into a chair. Below them a scooter coughed to life and puttered off down the lane. "And this time I could half answer it myself. But only half. I want the other half."

"What did he ask?"

"Not one shop on one day. All of them. On Tuesday. How many orders each kitchen got, and how much each one made." She pulled a face. "I know how to ask about one kitchen. But I don't want to run the same question seventeen times and write the answers on my hand."

Mohan smiled at the darkening sky. "No. You want to ask it once, and have it sort your whole city into piles for you. There's a

phrase for exactly that. It might be my favourite thing in the whole language."

He opened the laptop, and the screen threw a small blue glow between them on the terrace.

"Here's Tuesday," he said. "Every order, which kitchen, what it came to. Six orders, three kitchens, all mixed together, the way a day actually happens."

**ORDERS** · that Tuesday

| order_id | kitchen_id | total |
|---|---|---|
| 4402 | 17 | ₹370 |
| 4419 | 17 | ₹150 |
| 4431 | 17 | ₹170 |
| 4433 | 18 | ₹220 |
| 4440 | 18 | ₹120 |
| 4451 | 19 | ₹470 |

"Now watch. I'm going to ask the database to gather these into piles. One pile per kitchen. And then count how many orders are in each pile." He typed. "The phrase is GROUP BY. You tell it what to gather on. It does the sorting."

```
SELECT kitchen_id, COUNT(*) AS order_count
FROM orders
GROUP BY kitchen_id;
```

Meera read it slowly. "Group by kitchen_id. So it takes all the orders, and it makes a pile for each kitchen, and COUNT star counts how many are in each pile." She watched the answer appear: kitchen 17 with three, kitchen 18 with two, kitchen 19 with one. "It

sorted the whole day in one line. That would have taken me the back of an envelope and a lot of crossing out."

"And counting is only the start. Once you've made your piles, you can measure them any way you like." He added a line. "You want money too? Add up the totals in each pile. The word is SUM."

```
SELECT kitchen_id,
       COUNT(*)   AS order_count,
       SUM(total) AS revenue
FROM orders
GROUP BY kitchen_id;
```

"Count how many, and sum how much, for each kitchen, all at once." Meera leaned in as the numbers came back. Sri's pile: three orders, six hundred and ninety rupees. She sat back. "That's Sri's whole Tuesday. In one answer. Six ninety." She was quiet a second. "He'd be so pleased. He never knows his own numbers. He just cooks and hopes."

"You could tell him," Mohan said gently. "That's a thing you can do now. Walk into a man's shop and tell him true things about his own work that he never had a way to see."

Meera liked that thought so much she had to look away from it, out over the wall, at the lamps coming on across the neighbourhood.

"There's one ugliness left, though," she said, turning back. "Look. It says kitchen one seven. One eight. One nine. Numbers. If I show that to Sri he'll think I've forgotten his name." She looked at Mohan. "The names are in the kitchens table. But the orders are in the orders table. They're in two different rooms. How do I get them in the same answer?"

Mohan looked at her for a moment with something that was almost mischief and almost pride.

"You already know the shape of this," he said. "You built it yourself, on Sri's counter, weeks ago. Each order carries a KITCHEN_ID that points at the kitchens table. That pointer is a thread. Tonight you learn to pull it, and bring the two tables together into one."

"The foreign key."

"The foreign key. When you follow it to bring two tables together, it has a name of its own. A JOIN." He typed the word, and she watched how he wrote it, the orders table and the kitchens table named together, tied on the matching key. "Read the join out loud. It says: for each order, go and find the kitchen whose id matches, and treat them as one row."

```
SELECT k.name,
       COUNT(*)     AS order_count,
       SUM(o.total) AS revenue
FROM orders o
JOIN kitchens k ON k.kitchen_id = o.kitchen_id
GROUP BY k.name;
```

Meera ran it. And there it was, the whole of Tuesday, told properly at last. Not kitchen 17 but *Sri Idli*. Three orders. Six hundred and ninety rupees. Every kitchen by its true name, counted, summed, in plain sight.

"There," she breathed. "That's not a computer's answer. That's a human answer. I could print that and hand it to Dev and he'd just understand it." She looked at the little glowing table as though it were a photograph of someone she loved. "The orders and the kitchens were split into two rooms so nothing would ever be misspelled. And a join walks between the rooms and brings them together only when you ask. They get to be apart *and* together."

"That's the whole art of it," Mohan said. "Keep things in their own place, so each fact lives once and lives cleanly. Then join

them, at the moment of asking, into whatever picture you need. Apart for safety. Together for meaning."

· · ·

They sat with it a while, not talking. The city hummed below. Somewhere a television laughed to itself. The breeze finally came, the one the evening had been promising, and moved the warm air around them.

Meera realised she was completely happy, and that the happiness had no single cause she could point to. Some of it was the join. Some of it was Sri's six hundred and ninety rupees. Some of it, she did not examine too closely, was the plastic chair beside her own, and the fact that it was not empty, and that neither of them had said a word for a full minute and it had not felt like silence at all. It had felt like company.

"You've gone quiet," Mohan said, not opening his eyes.

"So have you."

"Mm." A pause. "It's a good roof for it."

It was. It really was. Below them the neighbourhood counted itself up, kitchen by lit kitchen, and Meera thought that she had spent her whole life until now with data she could not question and evenings she could not keep, and that both of those famines had ended in the same warm month, and she was not at all sure they were separate things.

WHAT MEERA LEARNED

**Grouping** turns SQL from single-row questions into business questions. GROUP BY gathers rows that share a value into piles, and **aggregate functions** measure each pile: COUNT (how many), SUM (add a column up), plus AVG, MIN, and MAX. The whole move is: make piles, then measure each pile.

A **join** pulls facts from two tables back together for a single question, matching rows on a shared key with an ON condition (typically a foreign key meeting the primary key it points to). This is the payoff of keeping each fact in one place: the data stays clean and un-duplicated, and you reunite exactly the pieces you need, only when you need them. It is the idea the whole relational world rests on.

One professional caution: a plain (inner) join keeps only rows that *match*, so entities with nothing to match, a kitchen with zero orders, silently disappear from the result. When the empties matter, an **outer join** keeps them and shows zero.

*Keep each fact in one place; join them when you ask. Make piles, measure the piles, and call things by their names.*

CHAPTER EIGHT

# Transactions

*The last biryani*

· · ·

The trouble came on a Friday, in the middle of the dinner rush, which is exactly when trouble prefers to come.

Meera and Mohan were on the terrace, half working, half not. Then her phone lit up with a message from Lakshmi. Lakshmi did not message often. When she did, it was short, and it was serious.

Two customers, Lakshmi wrote, had both been promised the same mushroom biryani. The last one of the night. There had been exactly one left. The app had cheerfully sold it twice. Now there were two paid orders, one biryani, and one very unhappy kitchen.

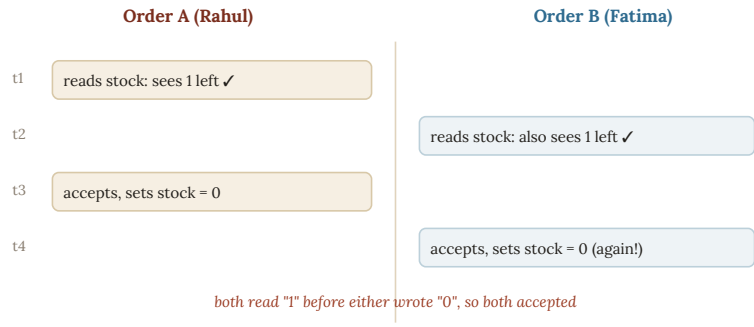
"Rahul and Fatima," Meera read off the screen, her stomach sinking. "Both got the same dish. Both got a happy little confirmation. And there's only one biryani." She looked up. "Mohan, how? The app checked the stock. It said one left. How did it sell one biryani to two people?"

Mohan did not laugh, and he did not look delighted. That was how she knew this one was serious.

"Because two people asked at the very same instant," he said. "And your app, being polite and a little naive, answered them both before either had finished. Come downstairs. This one you have to see drawn, or it won't feel real."

At the low table, he drew two columns. One for Rahul's order, one for Fatima's. And a clock ticking down the side.

"Watch the gap," he said. "The whole disaster lives inside half a second."




*The race. Both orders checked the stock before either had lowered it. Each honestly saw one biryani left, so each accepted. The reading and the writing were not glued together.*

Meera followed it down, and her face changed as she saw it.

"Rahul's order reads the stock. One left. Good," she said, tracing it with her finger. "But in the same breath, before Rahul's order has actually *taken* the biryani, Fatima's order reads the stock too. And it also sees one. Because Rahul hasn't taken it yet. So they both think it's theirs. They both say yes. They both set the stock to zero." She sat back. "They both read *one* before either wrote *zero*. They slipped through the gap."

"That has a name," Mohan said. "It's called a **race condition**. Two actions racing through the same gap, and both winning, which means everyone loses." He looked at her. "And it is not a rare, freak thing. On a busy Friday it is waiting to happen every single second. It is the most ordinary disaster in the whole trade."

"So how do you close the gap?" Meera asked. "How do you make sure only one of them can win?"

Mohan set the pen down, and his voice dropped into the low, careful register he used for the things that truly mattered.

"You stop treating *check* and *take* as two separate steps," he said. "You bundle them into one action. One thing that happens completely, or does not happen at all. With no gap in the middle for anyone to slip through." He wrote a word. "That bundle has a name. It's called a **transaction**."

"A transaction."

"A transaction holds the door. It says: from the moment I begin to check the stock, to the moment I take it, nobody else may touch this shelf. I check, I take, I close. All one motion. Then the next order may come. And by then the stock honestly says zero, and the second customer is told, kindly and correctly, that the biryani is gone." He wrote it out, plainly.

```
BEGIN;  -- open the transaction: hold the door

  -- take one biryani only if one is actually there
  UPDATE stock
  SET qty = qty - 1
  WHERE dish_id = 302 AND qty >= 1;

  -- if that changed no rows, there was none left: cancel everything

COMMIT;  -- all-or-nothing: make it real, and durable
```

"See the trick in the middle," he said. "It doesn't check the stock, and then, separately, take one. It does both in a single stroke. *Take one, but only if one is actually there.* If nothing was there to take, it changed nothing, and it knows to cancel the whole thing. There is no gap between the looking and the taking, because they are the same act."

"So one of them wins cleanly," Meera said, "and the other is told no, cleanly. Nobody gets a biryani that doesn't exist."

"Nobody gets a biryani that doesn't exist. Which is a decent rule for a company to live by."

"Does a transaction have rules of its own?" Meera asked. "Things it always promises?"

"Four of them," Mohan said. "People remember them by their first letters. It spells a word. But forget the word. Just hear what the four promises actually are." He wrote them out, one plain line each.

### THE FOUR PROMISES · ACID

| promise | in plain words | tonight's biryani |
|---|---|---|
| **Atomicity** | all steps happen, or none do | check-and-take is one act, never half-done |
| **Consistency** | the rules are never broken | stock can never go below zero |
| **Isolation** | concurrent work doesn't tangle | Order B can't peek mid-way through Order A |
| **Durability** | once done, it stays done | an accepted order survives a power cut |

*A transaction gives you all four at once. Money, stock, seats, anything that must not be sold twice, lives or dies by these.*

"All or nothing, never half," Mohan said. "The rules are never broken. Two orders never trip over each other. And once it's done, it stays done, even if the power dies a second later." He looked at her. "Four promises. Together they are why you can trust a database with something as fragile as the last biryani on a Friday night."

Meera read them twice. Then she reached for her phone, because a real customer was really waiting.

What happened next happened fast.

She called Lakshmi. Mohan, beside her, sketched the fix she would need Arjun to build into the app tonight. She talked. He wrote. She read over his shoulder and folded his notes into her words to Lakshmi, so the kitchen and the code stayed in step. Neither of them discussed who would do which part. It simply divided itself between them, the way work divides between two people who have stopped needing to explain themselves.

Refund Fatima the fair way. Give her the next mushroom biryani free, with a message in Meera's own words, not the app's. Let Rahul keep the one that truly existed. And tonight, before the next Friday, close the gap for good.

When it was done, and Lakshmi was calmed, and Fatima had written back a soft *oh, thank you, that's so kind*, Meera put the phone down. She found she was slightly out of breath, and not only from the emergency.

"We just did that without talking," she said. "You knew what I'd say to Lakshmi. I knew what you were writing before you finished. We just... moved."

Mohan looked at her, and for once he did not have a clever line ready. He looked as though the same thing had struck him, and struck him harder than he expected.

"We did," he said simply.

· · ·

The rush went on outside. Scooters came and went. Somewhere in the city, Fatima was being handed a hot biryani she had almost not received. Rahul was eating his, happily ignorant of how close it had come.

And on the low table, the little transaction sat, holding its door. From now on, the last biryani of any night would go to exactly one person, completely, or to no one at all.

Two months ago, Meera had been four founders and a spreadsheet, drowning. Tonight she had closed a gap that could sell one dish twice. And she had done it beside someone who no longer needed her to finish her sentences.

She did not yet have a word for the second thing. She was not sure she wanted one. Some shapes are better left, a while longer, unnamed.

WHAT MEERA LEARNED

When two things happen at nearly the same time, a naive "check then update" can go wrong, because another action slips into the gap between the checking and the updating. This is a **race condition**, and it is how one biryani gets sold twice.

A **transaction** bundles several steps into one all-or-nothing act, and the database promises not to let anything wedge itself in the middle. Transactions keep four promises, remembered as **ACID**: *Atomicity* (all steps or none), *Consistency* (rules never broken), *Isolation* (concurrent work doesn't tangle), and *Durability* (once done, it survives crashes). Anything that must never be sold twice, money, stock, a seat, depends on them.

In a real job: the moment your system has more than one user acting at once, race conditions are waiting. "Wrap it in a transaction" is one of the most common and most important fixes you will ever apply.

*When two hands reach for the last biryani,*
*the database must let only one of them close.*

PART TWO

# A Bigger Kitchen

*Zaayka is loved across the city now. The single database that carried it this far begins to strain, and Meera discovers there is a second kind of data work entirely: not running the business, but understanding it. Somewhere in these weeks, without either of them noticing the day it happened, the lessons stop being the reason they meet.*

· · ·

CHAPTER NINE

# Indexing

*The index at the back of the book*

. . .

The app had started to feel slow, and Arjun noticed first.

Arjun was the one who had built the app, back at the very beginning, in a single caffeinated month. He was one of the four partners, quick and clever and a little too proud of his own work, in the way of people who have not yet been humbled by their own success. When something he had made began to misbehave, he took it personally.

"It's the customer screen," he told Meera, dropping into the chair across from her at the little office they now rented, two rooms and a kettle. "When someone opens their past orders, it used to appear instantly. Now it takes a second. Two, sometimes. And it's getting worse every week." He rubbed his face. "I can't see why. The code hasn't changed. Nothing's changed."

"Something's changed," Meera said. "We have thirty times the orders we had two months ago."

Arjun looked at her, a little surprised. Three months ago, Meera would not have said that. Three months ago, Meera thought a database was a kind of spreadsheet with airs. Now she had said the true thing before he had, and they both noticed it.

"Ask Mohan?" Arjun said, half a question, half a surrender.

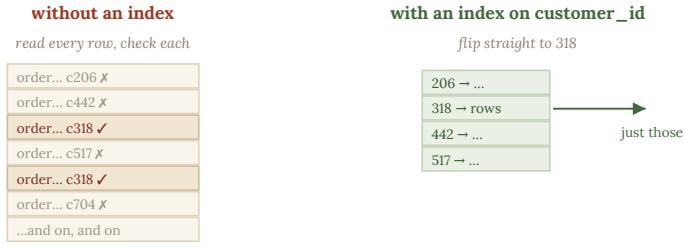
"I'll ask Mohan. But let me look first."

She did look first, because that had become a small private point of pride.

She opened the database and asked it for one customer's orders. Priya's. Then she asked the database a second question, a question about itself: *how did you find those*? The answer made her sit up.

To find Priya's handful of orders, the database had read every order in the whole company. All of them. One by one. Checking each: is this Priya's? No. Is this? No. Is this? Yes. And on, and on, through thousands of rows, to find a few.

That evening, she drew it for Mohan the way he always drew things for her.




*An index is a sorted lookup kept beside the table. Instead of reading every row, the database finds the value in the index and jumps straight to the matching rows.*

"It reads every single row," she said. "Every time. To find six orders, it looks at six thousand. That's why it's slow. And it'll only get slower, because every week there are more rows to wade through." She sat back, pleased with herself. "That has a name, doesn't it? Reading everything to find a few?"

"It's called a **full scan**," Mohan said. "The database, starting at the first row and reading all the way to the last, checking every one." He was not hiding his pride tonight. "And you didn't just spot that it was slow. You asked the database *what it did*. Most people never think to. They just complain. You opened it up and looked."

"So how do we fix it?" she said. "How does a database ever find anything fast, if not by reading everything?"

"The same way you'd find one word in a thick book without reading the whole book." He tapped the fat notebook she now carried everywhere. "You don't start on page one. You turn to the back. To the index."

"The index. The list at the back, with every important word and the page it's on."

"Exactly that. Every important word, listed once, in order, with the exact page beside it. You find your word in the index, and you jump straight to its page. You skip everything else." He looked at her. "A database can keep an index too. And it's the same idea, precisely."

"So we give the database an index. Like the back of a book."

"You tell it: keep me a sorted list of every customer, and beside each one, exactly where that customer's rows live. Then, when someone asks for Priya's orders, it doesn't read the whole company. It jumps to Priya in the index, and goes straight to her rows." He wrote it out. "One line. It turns a slow company into a fast one."

```
CREATE INDEX idx_orders_customer
    ON orders (customer_id);
```

"Create an index on the customer," Meera read. "And after this, asking for Priya's orders will be fast. Because it can jump to her, instead of reading everyone." She ran it. Then she asked for Priya's

orders again. The answer came back so fast there was no waiting at all. Just the answer, sitting there, as if it had always been there. She laughed out loud. "Oh, that's lovely. It's instant. Arjun's going to cry with relief."

"There's a catch, though," Mohan said. "There's always a catch, and the catch is where the real engineering lives."

"Tell me the catch."

"An index is not free. It's a second thing the database has to keep, beside the table. It takes up room. And every time you add a new order, the database must update the table *and* the index, both. So writing gets a little slower, to make reading much faster."

"So you don't index everything."

"You don't index everything. You index the things you search by often. It's a trade, made on purpose, every time. Fast to read, or fast to write. Knowing which one a particular question needs, that judgement, made a thousand times, is a good part of the job."

. . .

Later, walking Mohan down to his scooter, Meera remembered the thing she had been saving.

Because she had started doing that, lately. Saving things. All week, when something funny or strange happened with the data, she would notice a small instinct in herself: *save that, tell Mohan.* A message she almost sent a dozen times a day, and mostly didn't, because it was nicer to keep them, and hand them over in person, and watch him laugh.

"Arjun named a test kitchen after himself," she said at the door. "In the real database. It's kitchen ninety-nine. It sells one dish. The dish is called *Arjun's Genius*. It's priced at one crore rupees."

Mohan laughed the whole way down the stairs. The real laugh, the one she had started to work for on purpose.

Meera stood at the top and thought about the little collection she had now. A private index of things kept only because they were his to be told. She had not asked herself why she kept it. Some indexes you build without deciding to. You only notice, later, how much faster your whole day turns when he is the thing it points to.

WHAT MEERA LEARNED

As a table grows, a query that searches without help must read every row, a **full scan**, which gets slower and slower as data grows. An **index** fixes this: it is a sorted lookup kept beside the table (exactly like the index at the back of a book) so the database can jump straight to the rows it wants instead of reading everything.

But an index is not free. It costs disk space, and it makes writes a little slower, because every change must also update the index. So the craft is to index the columns you search by often (those in WHERE clauses and joins) and leave the rest alone. An index trades faster reads for slower writes.

In a real job: "the query is slow" is answered, nine times out of ten, by "it's missing an index" or "it isn't using the index it has." Knowing when *not* to add one is just as valuable.

*Don't read the whole book to find one word.*
*Keep an index, and turn straight to the page.*

CHAPTER TEN

# Two Kinds of Questions

*Why the app and the analyst fight*

. . .

There was a quiet fight going on inside Zaayka, and it was slowing everyone down.

On one side was the app. The app had to be fast. A customer tapping to order dinner would not wait three seconds. They would put the phone down and cook at home, and Zaayka would have lost them. So the app touched the database constantly, in tiny, quick bites. Save this order. Fetch this customer. Mark this one delivered.

On the other side was Dev. Dev had discovered reports.

Dev now checked a dashboard three times a day, the way some people check the weather. Revenue by kitchen. Orders by hour. Which neighbourhood went quiet on Tuesdays. He loved it. It had turned his worrying into something he could hold.

But every time Dev ran one of his big questions, it swept across every order the company had ever taken. And for those few seconds, the whole database groaned. The app went slow. And a customer somewhere put their phone down.

"So Dev's reports are hurting the app," Meera said. "And if I tell him to stop, I take away the thing that finally lets him sleep."

. . .

She went to Mohan's office this time, which was new. It was a tall glass building on the other side of the city, all cold air and quiet carpet. She felt small walking in, until she saw him wave from across a sea of desks, entirely himself, a fixed point in all that grey.

"You've got two questions," he said, once she'd laid it out, "that want completely opposite things from the same poor database. And you've felt the collision yourself. Let me show you the shape of it."

He drew two columns.

### TWO KINDS OF WORK

| | OLTP · running the business | OLAP · understanding it |
|---|---|---|
| what | taking orders, saving, updating | reports, trends, analysis |
| touches | a few rows, very often | millions of rows, now and then |
| needs | to be instant | to chew through volume |
| who feels it | the customer, live | you, reading a report |

**OLTP** = *online transaction processing (the app).* **OLAP** = *online analytical processing (the analysis). Same data, opposite demands.*

"On the left, running the business," he said. "Tiny, fast, constant. Take an order, save it, fetch it. It touches a few rows, and it must be instant, because a live human is waiting." He tapped it. "That kind of work has a name. People call it **OLTP**. Forget the letters. Just think of it as the shop floor. Busy, quick, never still."

"And Dev's reports?"

"The other kind. Understanding the business. That one is called **OLAP**. Again, forget the letters. Think of it as the back office, where someone sits with the whole year's books and adds things

up. It touches millions of rows, but only now and then. And nobody's tapping a phone, waiting. It doesn't need to be instant. It needs to chew through enormous piles without falling over." He looked at her. "Two completely different appetites. And right now you're feeding both from one plate."

"So they shouldn't share," Meera said slowly. "The fast little questions and the huge slow ones. They shouldn't live in the same place at all."

Mohan went still, that delighted stillness she had learned to watch for. "Say that again."

"They shouldn't share a house. The app needs one kind of place, built to be fast. Dev's reports need another kind, built to be huge. Trying to make one database do both is why everyone's unhappy." She stopped. "That's the answer, isn't it. Two places, not one."

"That is the answer," Mohan said. "And it's one of the biggest ideas in the whole field. You separate them. You let the app keep its fast little database for running the business. And you build a second, different place, shaped completely differently, just for understanding the business. A place where Dev can ask the hugest question he likes, and never slow down a single customer's dinner."

"And that second place has a name," Meera guessed. "Everything you love has a name."

"It's called a warehouse. A **data warehouse**." He said it the way you name a city you're about to take someone to for the first time. "And building your first one is where you stop being a person who keeps a database, and start being something bigger. It's most of the rest of what I know, Meera. It's the whole second half of the craft."

Meera felt the size of it settle over her. Not frightening. Just large, the way standing at the foot of a hill you mean to climb is large.

"Then that's next," she said.

"That's next." He glanced around at the cold glass tower, the grey carpet, the sea of silent desks. For a moment something crossed his face that she could not read. "You know, I build these all day, in here. Big ones. For a company so large the reports are about other reports." He turned back to her, and the look was gone, replaced by the usual warmth. "Somehow it's more fun explaining one to you, across a low table, over your terrible chai, than any of it is in here."

"My chai is excellent."

"Your chai is a small crime, and I keep coming back for it anyway. Draw your own conclusions." He said it lightly, a joke, entirely deniable, and turned to shut down his machine.

But Meera walked back out through the sea of grey desks with the sentence warm in her chest. She turned it over, and decided not to draw any conclusions at all. Some conclusions, once drawn, cannot be undrawn. And she was not ready yet to lose the not-knowing.

WHAT MEERA LEARNED

There are two fundamentally different kinds of work you ask of data. **OLTP** (running the business) means many small, fast touches, saving an order, fetching a customer, and it must feel instant because a user is waiting. **OLAP** (understanding the business) means reading huge numbers of rows at once for reports and trends, run only now and then.

They have opposite needs, so when a heavy analytical query runs on the same database that is taking orders, they fight, and the live app slows down. The fix is to give them separate homes: keep the app on its own database, and do analysis in a second place built for heavy reading, a **data warehouse**.

In a real job: "never run big analytics queries directly on the production database" is a rule you'll hear early and for good reason. The warehouse exists precisely so the business's questions don't slow the business down.

*Let the kitchen cook and the accountant count,*
*but give them separate rooms.*

CHAPTER ELEVEN

# The Warehouse, and Getting Data In

*A room just for thinking*

· · ·

The rain came without warning, the way it does in Bengaluru. A mild evening turned into a waterfall in the space of a breath.

Mohan had come over to help her build the warehouse. The second place. The room just for thinking. They had worked late. And when he got up to leave, the sky opened, and the lane below turned to a river, and the streetlights blurred into long gold smears through the running window.

"You're not riding in that," Meera said. "You'll die, and then who explains the warehouse to me."

"Your concern is deeply touching."

He had been caught in the first of it on the stairs, and he was wet through the shoulders. Without quite thinking about it, Meera found the largest thing she owned. An old soft T-shirt from a college fest, six years gone. She held it out to him.

"It's dry," she said. "You can't sit there dripping. You'll catch cold and blame my chai."

He took it. Their hands did the ordinary handoff. And then neither of them quite let go of the folded cloth. Just for a moment, a half-second longer than the handoff needed. The rain roared.

The lamp buzzed. They both looked up at the same instant. And the look held one beat too long to be nothing.

Then Meera turned very quickly to the window, and Mohan turned very quickly to change, and the moment folded itself away, unmentioned, into the loud wet dark.

· · ·

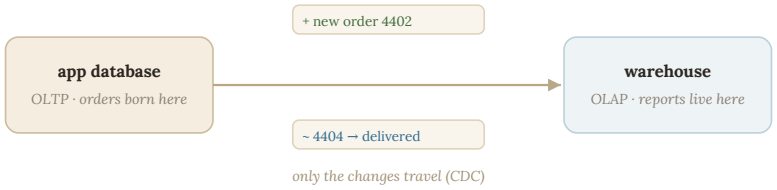
When he came back in the too-big T-shirt, looking younger in it somehow, they did not speak of the rain, or the pause. They spoke of the warehouse, gratefully, because the work was the safe place they could both stand.

"So the orders are born in the app," Meera said, a little too briskly, pulling the laptop between them. "How do they get to the warehouse? Do I just copy everything across every night?"

"You can. Many people do, at the start. Copy the whole thing on a schedule. It's simple, and it works." He tucked one foot under himself, at home again. "But think about the waste. Most of your orders don't change after they're placed. Copying all four hundred thousand every night, just to catch the few hundred that are new, is like repainting a whole house because you moved one picture."

"So you only carry what changed."

"You only carry what changed. A new order arrives. An order flips to delivered. Those changes, and only those, travel from the app to the warehouse." He drew it. "It has a name. It's called **change data capture**. The app keeps a little log of everything that happens, and you let that log carry just the news across, instead of copying the whole world each night."




*Change Data Capture carries only what changed, a new order, a status flip, from the app's database into the warehouse, instead of recopying everything.*

"Only the news travels," Meera said. "Not the whole newspaper, reprinted every day. Just today's headlines, added to what's already there." She watched the little diagram, the new order crossing the gap alone. "It's kinder. To the app, to the network, to everything."

"It's kinder, and it's faster. And at scale, it's the difference between possible and impossible."

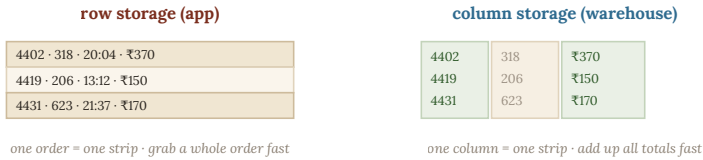
"Now, the warehouse itself," Mohan said. "It's shaped differently inside from the app. And the difference is beautiful, once you see it. May I ruin storage for you forever?"

"Please."

"The app stores an order as one strip. Everything about order 4402 sitting together in a row, so it can grab the whole order in one snatch. Perfect for the shop floor, where you want one whole order, fast." He drew the two shapes side by side. "This way of storing, everything about one order kept together in a row, is called *row storage*."

"And that's not good for Dev's reports?"

"Think about what Dev actually wants. He never wants one whole order. He wants one *column* across a million orders. The total, of every order, to add them all up. And in row storage, those totals are scattered, one in each strip, a million tiny reaches."




*The same orders, two ways. Rows keep each order together (good for the app). Columns keep each field together (good for adding one column across millions of rows).*

"So the warehouse turns it sideways," Meera said, staring. "It stores all the totals together. A whole column, side by side. So when Dev wants to add up every total, they're already lined up, ready to be swept up in one motion." She looked up. "That's the other way of storing. By column."

"Column storage. And there's your sentence for it." Mohan put a hand to his chest as if wounded. "Row storage keeps an *order* together. Column storage keeps a *question* together. That's better than how I say it. I'm going to steal it and never credit you."

"You steal my chai and my sentences. A woman has to watch her things around you."

He laughed, and it was easy again, the pause from the doorway smoothed over by the work. And they built her first warehouse together, while the rain kept the rest of the city out.

· · ·

The rain did not stop for a long time. Neither of them suggested that it needed to. They sat in the lamplight with the storm sealing the windows, and worked, and did not work, and talked about nothing. It was, Meera thought, the nicest evening she could remember. And she was almost cross about how nice it was.

When the rain finally thinned near midnight, he did go. He went in the borrowed T-shirt, promising to return it, his wet shirt in a bag. Meera stood at the window and watched the single headlight wobble off down the drying lane. Then she tidied the flat, slowly, humming. She was in no hurry at all to reach the part of the night where he was gone.

She did not think the word for it. She had become very good, this season, at not thinking the word for it. But she left the lamp on a while after there was any reason to. The flat still smelled faintly of rain, and of him. And she was in no hurry, no hurry at all, to turn it off.

WHAT MEERA LEARNED

Getting data from where it's born (the app) to where it's analysed (the warehouse) is called **ingestion**. The simple way is to copy everything on a schedule; the smarter way is **Change Data Capture** (CDC), which carries only what changed, a new order, a status flip, so the warehouse stays fresh without recopying everything.

A **data warehouse** stores data by **column** instead of by row. Because analytical questions usually need a few columns across many rows (add up every total), columnar storage reads only the columns it needs and flies. Row storage suits the app (grab a whole order); columnar suits analysis (grab one column across everything).

You rarely build this yourself: rented warehouses like Snowflake and BigQuery do the columnar work for you, and you drive them with the same SQL you already know.

In a real job: a huge share of data engineering is building and maintaining reliable ingestion, moving data cleanly from source systems into a warehouse, day after day, without loss or duplication.

*Store data the way you'll ask about it.*
*By row to live it, by column to understand it.*

CHAPTER TWELVE

# ETL, ELT, and Batch

*Carrying data across, and cleaning it on the way*

. . .

Anjali had a problem, and it was making the reports lie.

Anjali ran Zaayka's marketing now, the work Meera used to do before the data swallowed her whole. She was warm and fast-talking and ran three phones at once. And she had noticed something wrong in the warehouse. The customer names were a mess.

The same person appeared as PRIYA, and Priya, and priya s. A phone number had spaces in one place and none in another. And because of it, Dev's lovely new reports were quietly counting some customers twice.

"It's the old wound," Meera told Mohan that evening. "Sri's five spellings, all over again. But this time it's in the customers. The app lets people type their names however they like. So the same Priya arrives in the warehouse as three different Priyas."

"Then somewhere between the app and the report," Mohan said, "the mess has to be cleaned. The only question is where. And that question has a famous answer, and a famous argument. You're about to learn both."

"Every time you carry data from where it's born to where it's studied," he said, "you're really doing three jobs, not one." He held up three fingers. "First, you take the data out of the app. That's called *extract*. Second, you clean it and reshape it, fix the spellings,

tidy the phone numbers. That's *transform*. Third, you put it into the warehouse. That's *load*. Extract, transform, load."

"Three jobs. Carry, clean, put away."

"And here's the whole argument of the field, in one small choice. What order do you do them in?" He drew two rows. "The classic way cleans the data *before* it enters the warehouse. Extract, then transform, then load. You only ever let tidy things in the door. Very neat. People call it **ETL**, after the order of the three words."

### TWO ORDERINGS OF THE SAME THREE JOBS

| | **ETL · the classic way** | **ELT · the modern way** |
|---|---|---|
| order | extract → transform → load | extract → load → transform |
| clean | before entering the warehouse | inside the warehouse |
| suits | when the warehouse is precious | when the warehouse is cheap and strong |
| keeps the raw? | often no | yes, the raw is loaded and kept |

*Same three jobs. The only question is whether you clean before loading (ETL) or after (ELT). Modern teams often prefer ELT, and keep the raw data too.*

"And the modern way?"

"The modern way says the warehouse is now so cheap and so powerful that you needn't clean on the doorstep. Let everything in, mess and all, then clean it once it's *inside*. Extract, load, transform. That order is called **ELT**. Two little letters swapped. And it changes how half the world builds these things."

Meera thought about it, chin on her hand.

"I think I like ELT," she said slowly. "Because... if you clean before you load, you throw the mess away at the door. And what if,

next year, the mess was telling you something? That people who type in all capitals are older, or angrier, or something you didn't know to look for yet?" She looked up. "If you cleaned it away at the door, it's gone. But if you keep the raw mess in the warehouse, you can always go back to it."

Mohan did not say anything for a moment. He had the look of a man who set a trap for a rabbit and caught something far more interesting.

"Keep the raw," he said quietly. "That's a senior instinct, and most people take five years to feel it. Never throw away the original just because you've made a tidy copy. The tidy copy answers today's questions. The raw original answers the questions you haven't thought to ask yet." He tapped the table. "Clean a version. But keep the mess. The mess is where next year's discoveries hide."

"And all this carrying and cleaning," Meera asked, "does it happen all the time? Every second?"

"Usually not. Usually it happens in *batches*." He saw her not quite follow, and explained. "A batch is a big scheduled gulp. Once a night, say, while the city sleeps, the whole thing wakes up. It carries the day's changes across, cleans them, loads them, and goes back to sleep. Big, tidy, on a timetable. That's **batch**, and for most of what a company needs, it is exactly enough." He stretched. "There's a faster, live way, for when a nightly gulp isn't quick enough. But that's a story for when we have riders on the road, and a map that needs to move. Not yet."

"Riders on a map," Meera said. "You keep doing that. Dropping a hint, then closing the door on it. A faster way. A map that moves. You leave them half open on purpose."

"I do it on purpose." He grinned. "Keeps you turning up."

"I turn up because you bring biscuits."

"I bring biscuits because you turn up. Seed, tree. Extract, load. Some orderings we never do settle." He said it easily, gathering his things. And it was only a joke about a pipeline. And it was also, she felt, not only that.

· · ·

Here is the small true thing Meera did that night, and told absolutely no one.

The borrowed T-shirt had come back three days after the rain. Washed, folded, smelling of a soap that was not hers. She had meant to put it in the cupboard. She had not. It had migrated, by a route she declined to examine, to the top of the pile she actually wore. And tonight she put it on to sleep. The old soft college cotton, too big for her. She did not ask herself why. She had become so very good at not asking.

She had cleaned a tidy version of her feelings. The friendly version. The deniable version. The one she showed the world. But she had kept the raw. She lay in the dark, in the too-big shirt, and understood, dimly, that she was keeping the raw, for a question she had not yet found the courage to ask.

*The mess is where next year's discoveries hide.* She fell asleep before she could decide whether that frightened her or not.

WHAT MEERA LEARNED

Moving data is rarely just moving; it usually needs cleaning too. Nearly every pipeline does three jobs: **Extract** (take data from the source), **Transform** (clean and reshape it), and **Load** (put it in the warehouse). Cleaning before loading is **ETL** (the classic way, when the warehouse was precious). Cleaning after loading, using the warehouse's own power, is **ELT** (the modern way), which also keeps the raw data so your cleaning becomes a recipe you can re-run and improve.

**Batch** processing means handling a pile of data all together on a schedule (like a nightly copy). It is simple and sturdy and how most data work still happens, but it has an opposite, continuous, real-time flow, which we'll meet later.

In a real job: "we're an ELT shop" or "that's a batch pipeline" are sentences you'll hear constantly. Keeping the raw data (ELT) is now a widely followed good habit, precisely because it lets you fix mistakes without losing anything.

*Extract, transform, load, in whichever order suits.*
*And keep the raw, so you can always begin again.*

CHAPTER THIRTEEN

# Dimensional Modelling

*The star in the middle*

. . .

Dev wanted to slice the world a hundred different ways, and the warehouse was making him work too hard for it.

He came to Meera with a wish list, shy about it, the way he got when he wanted a lot. Revenue by kitchen. By area. By day of the week. By new customers, and by old ones. And all of those crossed together. But every one of his questions meant a different tangled join, and Dev, who was not a data person, kept getting lost in the tangle and giving up.

"He shouldn't have to think this hard to ask a simple thing," Meera told Mohan. "There has to be a shape that just lets him slice. Any way he wants. Without a fight."

"There is," Mohan said. "It's one of the loveliest shapes in the whole craft. And it's built out of one simple idea, so let me give you that idea first, on its own, before anything else."

"Start with what Dev actually measures," Mohan said. "The hard numbers he adds up. Revenue. The count of orders. Those are the facts. The *what happened*." He wrote them down. "Now. Every one of Dev's questions takes those numbers and looks at them from some angle. Revenue *by kitchen*. Orders *by day*. Revenue *by area*."

"By kitchen. By day. By area," Meera repeated. "The angles."

"Yes. And that word, angle, is the whole idea. Each way of slicing, by kitchen, by day, by area, by customer, is called a **dimension**." He said it slowly, and let it sit. "A dimension is just one angle you can look from. One way of asking about the numbers. Kitchen is a dimension. Day is a dimension. Area is a dimension. That's all the word means. An angle."

Meera wrote it down. *Dimension: an angle you look from.* It was a small, calm word now that she had it.

"So Dev has numbers in the middle," she said, "and a set of angles he wants to look from."

"Exactly. And now we give each of those two things its own kind of home."

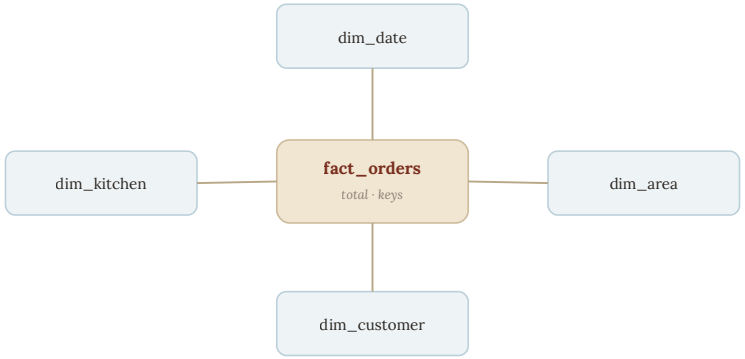
"The numbers first," Mohan said. "You put the hard numbers, and nothing else, in one table at the centre. One row for every order. Just the measurements, and the keys that point outward to the angles. This centre table has a name. It's called a **fact table**. Because it holds the facts. The *what happened*."

"Just the numbers. In the middle."

"Just the numbers, in the middle. And then, around it, one table for each angle. A table for kitchens. A table for areas. A table for dates, so Dev can ask about Tuesdays, or Diwali, or last March. A table for customers." He began to draw. "Each of those angle tables is called a *dimension table*. Because each one holds a dimension. One angle."

"Fact table in the middle. Dimension tables around it."

"Fact in the middle. Dimensions around it. And when you draw the lines from the centre out to each one..." He turned the notebook toward her.




*A star schema. The fact table (the measurements) sits in the centre; the dimension tables (the angles you slice by) surround it like points of a star.*

"It's a star," Meera said, looking at the shape.

"It's a star. People call it a **star schema**, for exactly that reason. Fact table burning in the middle. Dimension tables around it like the points of a star." He looked at her. "And here's why Dev will weep with joy."

"Because now," Meera said slowly, following it, "any question he has is simple. Pick the numbers in the middle. Then pick which points of the star to slice by. Revenue, by kitchen and by date. Orders, by area. He never has to untangle anything. He just points at the middle, and points at the arms he wants."

"He points at the middle, and the arms he wants." Mohan smiled. "You've got it. That's the whole gift of the shape. Every question becomes: which numbers, sliced by which angles."

Meera looked at the star for a while. Then a small frown crossed her face.

"Can I ask about something that feels wrong?" she said. "Back at the very start, you taught me to keep every fact in one place. Nev-

er repeat anything. And this feels like it might repeat. The kitchen's name sits in the kitchen dimension. But if I wanted to, couldn't I have just put the name straight in the big fact table, next to each order?"

"You could," Mohan said. "And now you've found the one real tension in all of this, so let me name it clearly, on its own."

"At the start, for the app, we were strict," he said. "One fact, one home, never repeated. That keeps the app safe when things change. That strictness has a name, we call it *normalising*, but forget the word. Just remember: one fact, one home."

"One fact, one home. I remember."

"In the warehouse, for reports, we relax that rule, a little, on purpose. We let some things sit closer to the numbers, even repeated, so the questions run fast and stay simple. That deliberate relaxing has a name too. It's called *denormalising*." He held up a hand. "But hear the idea, not the word. In the app, you organise for *safety*, because things change. In the warehouse, you organise for *questions*, because things are mostly just being read. Two different jobs. Two different shapes. Same data."

"Safety in the app. Questions in the warehouse," Meera said. "So it's not that the old rule was wrong. It's that the warehouse has a different job."

"A different job. That's exactly it. You're not breaking the rule. You're choosing the right shape for the work." He looked pleased. "Most people never see that clearly. They think one way must be right and the other wrong. You saw it in one go. It's the same data, wearing the shape each job needs."

They finished Dev's star late that night. And it was, Meera had to admit, a beautiful thing. The fact table burning small and bright in the middle. The dimensions arranged around it like a constellation she had drawn herself.

"Give it to Dev tomorrow," Mohan said, pulling on his jacket. "Watch what he does when he realises he can slice any way he likes, and never fight the tangle again. He'll look like a man handed a new pair of eyes."

"He'll check it three times to make sure it's real. Then he'll believe it. Then he'll never go back."

"See, you finished the thought before I did." Mohan smiled, one hand on the door. "That's a good sign, in an engineer. When you can feel where a shape wants to go before it gets there. It means you've stopped learning the craft, and started to feel it."

He left. And Meera sat a while under the lamp, with the star she had helped draw. And she thought that there were two constellations in the room tonight. One was on the page. The other was the shape of their two voices, learning to move as one, each able now to reach the end of the other's sentence. She did not draw the second one. But she could feel its lines, bright and unspoken, filling the quiet flat.

WHAT MEERA LEARNED

Analytics data is best shaped as a **star schema**. Separate the measurements (the numbers you add up, revenue, counts) into a central **fact table**, one row per event, and the angles you slice by (kitchen, area, date, customer) into **dimension tables** around it. Fact in the centre, dimensions as the star's points.

In the warehouse, we often deliberately copy some data into the dimensions, **denormalisation**, the opposite of the "one fact, one place" rule for app databases. Both are right in their own place: normalise where data changes (to keep truth clean), denormalise where data is read (to make questions fast).

Every fact table has a **grain**: exactly what one row represents (an order? an item? a kitchen-day?). Decide the grain first and never mix grains, or reports start to lie.

In a real job: "what's the grain of this table?" is one of the most useful questions you can ask in any data review. Star schemas and clear grain are the backbone of trustworthy analytics.

*Put the numbers in the middle and the angles around them,*
*and always know what one row means.*

CHAPTER FOURTEEN

# Slowly Changing Dimensions

*When a kitchen changes its name*

. . .

A KITCHEN changed its name, and it broke a year of history in a single afternoon.

The kitchen was Paradise Dosa, over in the next area, kitchen number twenty-two. For years it had been a small dosa counter. Then the owner's son came home from Dubai with money and ideas, and the little counter became a full sit-down place. New sign. New name. Paradise Family Restaurant. Same kitchen, same number, same good dosas. Just a grander name over the door.

Meera did the obvious thing. She opened the kitchens table and changed the name in the one row where it lived. Paradise Dosa became Paradise Family Restaurant. Clean, one edit, exactly as Mohan had taught her years ago. One fact, one home.

Then Dev came to her, looking worried, holding a report.

"Something's wrong with Paradise," Dev said. "Look. It says they were a big family restaurant all last year. Huge tables, big orders, the lot. But that's not true. Last year they were a tiny dosa counter. They only changed a month ago." He frowned. "Your report is telling a lie about the past."

Meera looked, and her stomach dropped, because Dev was right.

Every old order from Paradise, orders from six months ago, back when it was a humble counter, now showed the new grand name. She had changed the one row where the name lived. And in doing so, she had reached back in time and quietly rewritten every order that pointed at it. The past now claimed Paradise had always been a family restaurant. It was clean. It was tidy. And it was false.

That evening she brought it to Mohan, and for the first time, she brought it not as a student, but as someone who had found a real crack in something.

"I did exactly what you taught me," she said. "One fact, one home. I changed the name in the one place it lived. And it broke the whole past." She looked at him. "Your rule was wrong."

Mohan opened his mouth. Then he closed it. And then, slowly, he smiled, and it was a different smile than his usual one. There was something almost proud in it, and something else too, something she could not quite read.

"Say that again," he said quietly.

"Your rule was wrong. Not wrong for the app. But wrong here. Because some facts aren't meant to have just one version. A kitchen's name isn't one fact. It's one fact *now*, and a different fact last year. And both are true, for their own time." She was working it out as she spoke. "When I crushed them into one, I lost the past. The report needs to know what Paradise was called *at the moment of each order*. Not just what it's called today."

Mohan sat back and looked at her for a long moment.

"You just caught me teaching you something too simple," he said. "And you caught it yourself, in the wild, on a real kitchen. That's not a student thing to do, Meera. That's the thing I do. That's the whole job."

"So how do I fix it?" she asked. "How do I let a name change, but keep the old name true for the old orders?"

"You stop overwriting. You start keeping versions." He pulled the laptop over. "The kitchen keeps its number, twenty-two, forever. But instead of one row for it, you allow more than one. One row for each version of its life. Each row says: here is what this kitchen was called, and here are the dates that name was true."

"So Paradise gets two rows. The counter, and the restaurant."

"Two rows. Same kitchen number. Different names, each with its own stretch of time." He drew it out.

**DIM_KITCHEN · TYPE TWO, KEEPING HISTORY**

| row_id | kitchen_id | name | valid_from | valid_to | current |
|---|---|---|---|---|---|
| **881** | 22 | Paradise Dosa | 2026-03-01 | 2026-06-30 | no |
| **882** | 22 | Paradise Family Restaurant | 2026-07-01 | NULL | yes |

*Same kitchen (22), two versions. Each order links to the row whose date range covers it. NULL valid_to means "still current."*

Meera read it, and it opened up in front of her.

"So the old orders point at the first row," she said, tracing it. "Paradise Dosa, valid for those early months. And the new orders point at the second row. Paradise Family Restaurant, valid from last month on. Each order finds the version that was true when it happened." She looked up. "The past stays true. The present stays true. Nothing gets crushed."

"And that last column, *current*," Mohan said. "It just marks which version is the one today. And the empty end-date, the NULL, on the newest row, means *still true, hasn't ended yet*. The same honest NULL you met a long time ago, saying *not yet*."

"It's the same NULL," Meera said, pleased to see an old friend. "Still meaning *we don't know the end yet*. Because the restaurant's name hasn't stopped being true."

"This whole pattern has a name," Mohan said. "A dimension that mostly stays still, but changes slowly over time, and whose history you choose to keep, is called a **slowly changing dimension**. Keeping the old versions like this, one new row per change, is the most common way to do it." He looked at her. "And you didn't learn it from me. You hit the exact problem it exists to solve, and you felt the shape of the answer before I said a word."

. . .

They fixed Paradise together, and it went quiet the way it did when the work was done and neither of them had moved to end the evening.

"Can I tell you something?" Mohan said. He was looking at the two rows of Paradise on the screen, not at her. "For years I've taught people this craft. And they learn it. They get good. But there's a moment, and it doesn't always come, where the student stops receiving and starts *seeing*. Where they catch something I missed, in the wild, before I do." He turned to her. "That was today. You stopped being someone I teach."

"What am I now, then?" Meera asked. She meant it lightly. It did not come out lightly.

Mohan was quiet a moment. The lamp hummed. The tulsi kept its small green watch.

"Someone I build things with," he said. "Which is rarer. And, it turns out, harder to walk away from at the end of an evening."

The words sat in the room. Meera looked at the two versions of Paradise on the screen, each true for its own time, and found she did not know what to say.

"Are we still talking about data?" she asked. Quietly. Not quite joking.

Mohan met her eyes, and for once did not reach for the easy deflection, the biscuit, the grin. He held the look one beat longer than safe.

"Mostly," he said.

And that *mostly*, that one small honest word, sat between them in the lamplight like a valid-from date with no end yet written. Neither of them reached to fill in the rest. Some records you leave open on purpose, because the story they hold is still, quietly, true, and not yet finished.

WHAT MEERA LEARNED

A **slowly changing dimension** is a mostly-still thing (a kitchen, a customer) whose details occasionally change. Handling the change with **type one** means overwriting the old value, simple, but it erases history and rewrites old reports. **Type two** keeps history: it closes off the old version and adds a new one, each with a date range, so every order links to whichever version was true when it happened. The past stays true; the present moves on.

Crucially, switching to type two only helps going forward. To make old data obey the new design, you must **backfill**: a one-time pass that reassigns historical records to the correct version by date. A fix that only works going forward is only half a fix.

In a real job: type-two dimensions and backfilling are everyday realities in analytics engineering. And no advice, not even from a senior, is above being checked; the best engineers verify the logic all the way down.

*Let names change without letting the past lie,*
*and remember: the history must obey the new rule too.*

PART THREE

# Many Kitchens

*Zaayka opens its second city, and then its third. The data grows beyond what any single machine can hold. It sounds frightening. It turns out to be the same ideas Meera already knows, only larger, and shared across many hands. And in the growing, she begins to want things she hasn't let herself name.*

· · ·

CHAPTER FIFTEEN

# Partitioning & Sharding

*When one table becomes many*

. . .

Nisha came in on a Monday and announced they were opening Hyderabad.

Nisha was the fourth of them, the one who watched the money and the growth. She guarded a spreadsheet of runway the way a dragon guards gold, and when she said the company was ready to grow, it was ready. Bengaluru was full of Zaayka now. The next city was Hyderabad. After that, if it worked, Pune.

Meera was thrilled, and then, an hour later, quietly frightened. Because she was the one who had to make the data survive it.

All four hundred thousand orders lived on one machine. One computer, holding the whole company. It had been enough. But three cities of orders, and then ten, would not fit on one machine, however large. She could feel the shape of the problem without knowing its answer.

So she took it to Mohan, on the terrace, on a warm evening the colour of weak tea.

"One machine has held everything so far," she said. "But it won't hold three cities. And it definitely won't hold ten. So what do I do? Buy a bigger and bigger machine forever?"

"You can, for a while," Mohan said. "You buy a bigger machine. Then a bigger one. People call that growing *up*, taller and taller.

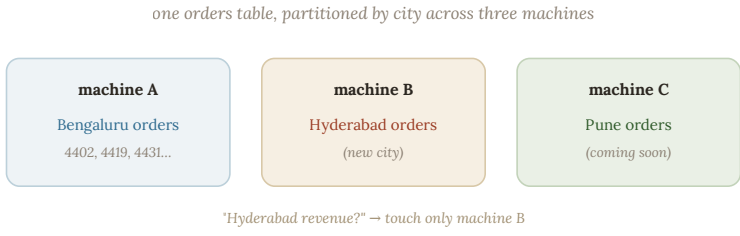
But there's a ceiling. One day there is no bigger machine to buy, and you're stuck." He tore a chapati from the plate between them, because he was always a little hungry. "So instead of growing up, you grow *out*. You stop trying to fit everything on one machine. You spread it across many."

"Spread it how, though? I can't just cut the orders in half randomly. I'd never find anything."

"No, not randomly. You cut it along a line that means something. And you've already half said the line yourself." He looked at her. "What's the most natural way to split Zaayka's orders into piles?"

Meera thought. "By city. Bengaluru orders, Hyderabad orders, Pune orders."

"By city. So you put Bengaluru's orders on one machine. Hyderabad's on another. Pune's on a third. Same orders table, same shape, just split across machines, each machine holding one city's slice." He drew it.




*Partitioning by city. Each machine holds one city's orders. A question about one city touches only that city's slice and skips the rest, called partition pruning.*

"Splitting a big table into slices like this, each slice kept separately, is called **partitioning**," he said. "Each slice is a partition.

You've cut one enormous table into city-sized pieces, and handed each piece to its own machine."

"Partitioning. By city." Meera looked at the three machines in the drawing. "And here's the part I like. If Dev asks about Hyderabad's revenue, the database doesn't have to touch Bengaluru or Pune at all. It just goes to the Hyderabad machine. It only reads the slice it needs."

Mohan pointed at her like she'd won something. "That's the whole gift. A good partition means most questions touch only one slice. Ask about Hyderabad, touch only Hyderabad. The other cities stay asleep. It's why partitioning makes huge data fast again, you stop reading what you don't need."

"Is there a catch?" Meera asked. "You taught me there's always a catch."

"You're learning to expect me. Yes, there's a catch, and you should hear it clearly." He set down the chapati. "Choose the wrong line to cut on, and you make things worse. Suppose you partitioned by day instead of city. Then a question like *Hyderabad's whole year* would have to visit every single day's slice, three hundred and sixty-five machines, to gather one city. You'd have made your common question slow instead of fast."

"So the art is choosing the cut," Meera said. "You cut along the line your questions ask about most. We ask about cities constantly. So we cut by city."

"You cut by how you ask. Always. The right partition is the one that matches your real questions." He smiled. "You said that better than the textbook, again. You keep doing that. It's becoming a habit."

· · ·

They built the plan for Hyderabad that night, slice by slice. And somewhere in the middle of it, Meera noticed something that had nothing to do with machines.

She noticed she was not frightened anymore. An hour with him, and a problem that had felt like a cliff had become a set of steps. It kept happening. It had been happening for a year. And she caught herself thinking a thought she did not examine too closely: that whatever Zaayka became, two cities or ten, she could not now picture building any of it without him in the chair beside her.

It was not a soft thought. It was a structural one. He had become, without her deciding it, load-bearing. One of those beams you don't notice until you imagine the house without it, and the whole roof comes down in your mind.

"You've gone quiet again," Mohan said, not looking up from the plan.

"Just thinking about how big this is getting."

"Mm." He added another slice to the drawing. "Big is only frightening alone. Split across enough hands, big is just... a lot of small." He glanced at her. "Same is true of most things, I've found. Not only data."

Meera did not answer that. But she added a slice to the plan of her own, and their two pens moved on the same page in the lamplight, and she thought that this, too, was a kind of partitioning. The load of a life, shared out, until the frightening size of it became a lot of small, carried between two people, and no longer heavy at all.

WHAT MEERA LEARNED

When data grows too large for one machine, you split it into pieces and spread them across several machines. Cutting a table into pieces is **partitioning**; when the pieces live on separate machines they're often called **shards**. It is the same "divide the work" instinct, scaled up.

You choose a **partition key** (like city) that matches how you ask questions, so that a question about one city touches only that city's slice, this skipping is **partition pruning**. Choose the key well and most questions stay small; choose it badly and every question touches every machine.

Some questions (total revenue across all cities) genuinely need every slice, gathered and combined, and coordinating work across many machines is one of the central hard problems of distributed data.

In a real job: partitioning by date or by tenant/region is everywhere, and "what's the partition key?" is a question that decides whether a system is fast or miserable at scale.

*When one table grows too heavy for one machine, cut it along the line you ask about, and share the load.*

CHAPTER SIXTEEN

# Distributed Computing & Spark

*Many hands make light work*

. . .

The data was spread across three cities now, and Dev's oldest question had stopped working.

It was a simple question. The company's most popular dish, across everywhere. It used to be easy, back when everything sat on one machine. Now the orders lived on three machines, one per city, and Dev's question just sat there, spinning, unsure how to count across all three at once.

"It can't add them up anymore," Meera told Mohan. She had called him late; it was nearly eleven. "Each city knows its own favourite. But nothing knows the favourite across all three. The counting has nowhere to stand."

"Because you're still thinking of the counting as one job, in one place," Mohan said. "When the data is spread out, the *work* has to spread out too. You don't bring three cities of orders to one counter. That's the old thinking. You send the counting *to* the data."

"Send the work to the data. Not the data to the work."

"That one sentence is most of it. Let me show you the shape. It comes in three simple moves."

"First move," Mohan said. "Each machine counts its own slice, on its own, at the same time as the others. The Bengaluru machine

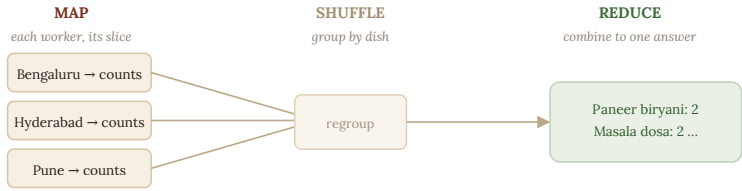
counts Bengaluru's dishes. Hyderabad counts its own. Pune, its own. All three working at once, nobody waiting. That first move, every worker handling its own slice in parallel, is called the **map**."

"So all three count at the same time. Three workers, three slices, one moment."

"Then the second move. Now each machine has a little pile of counts, but they're scattered. You need all the counts for *masala dosa* to end up in the same place, no matter which city they came from. So the machines pass their counts around, gathering all the same dishes together. That regrouping is called the **shuffle**."

"The shuffle. Everyone hands their dosa counts to whoever's in charge of dosas."

"Exactly that. And the third move. Now that every dish's counts are gathered in one place, you add them up, once, to a final number. That last combining is called the **reduce**." He drew all three moves, flowing into each other.




*Map, shuffle, reduce. Each machine counts its own slice (map), the partial counts are regrouped by dish (shuffle), then combined into one ranked answer (reduce).*

"Map, shuffle, reduce," Meera read. "Count your own slice. Regroup so like meets like. Combine to one answer." She followed the arrows. "And the beauty is the first move. All three cities count at

once. So three cities takes barely longer than one. If I had ten cities, ten machines would all count together, and it'd still be fast."

"That's the whole miracle," Mohan said. "The work grows, but you throw more workers at it, and they all labour at once. Ten cities, ten hands. A hundred, a hundred hands. The answer comes back in about the same time, no matter how big it gets." He sat back. "There are famous tools that do this for you, so you don't wire it by hand. But the idea underneath every one of them is exactly what you just said. Map, shuffle, reduce."

"Send the work to the data," Meera said. "And let many hands work at once."

"You've had it since the first sentence. The rest is machinery."

· · ·

The problem was solved. It was past midnight now. And neither of them mentioned the hour, or hung up.

The work was done, and the reason for the call was over, and the honest thing would have been to say goodnight. Instead the conversation just... kept not ending. They drifted from dishes to nothing in particular. A thing Nisha had said. Whether Sri's shop should be on the new city's launch poster. Whether Mohan's giant glass company was as soulless as it sounded. It was, and he described a meeting about a meeting until Meera laughed helplessly into the dark of her flat.

"It's half past twelve," she said finally, not because she wanted to hang up, but because someone had to notice.

"It is."

"We solved the dish thing an hour ago."

"We did."

A silence. Not an awkward one. The comfortable kind, the kind that is its own small conversation. Down the phone she could hear him not hanging up, and she knew he could hear her not hanging up, and the two not-hangings-up leaned toward each other across the sleeping city like two machines holding the same open line.

"Goodnight, Meera," he said at last. Softly. As though it cost him something.

"Goodnight."

Neither moved. The line stayed open one more breath, two, three, each of them listening to the other simply be there, in the dark, at the far end of the line. Then, gently, the call ended, and the flat was very quiet, and Meera lay looking at the ceiling for a long time, holding the shape of a silence that had felt like being held.

WHAT MEERA LEARNED

When there's simply too much data to chew through on one machine, you split the *work* across many machines: **distributed computing**. The classic shape is **map** (each machine handles its own slice and makes a partial answer) then **reduce** (the partial answers are combined into one).

Between them sits the **shuffle**: moving partial results across the network so related data (all the masala-dosa counts) lands together to be combined. The shuffle is usually the most expensive part, so a core skill is arranging work to shuffle as little as possible.

**Spark** is the industry workhorse that does this coordination for you: you describe the question in familiar grouping-and-joining terms, and it splits the work across a cluster, handles the shuffle, recovers from failures, and returns the answer. It is your GROUP BY, scaled across many machines.

In a real job: Spark (and its ideas) power a huge fraction of large-scale data processing. "Reduce the shuffle" is one of the most common performance conversations you'll have.

*Too much for one pair of hands?*
*Give everyone a tray, then add up the tallies.*

CHAPTER SEVENTEEN

# Cloud & Object Storage

*Someone else's computer*

. . .

The machines were becoming a problem that money alone could not fix.

Zaayka now owned computers. Real ones, humming in a rented room across town, bought with Nisha's careful money. One per city, plus spares. And they were a headache. When a big report ran, the machines strained and everything else slowed. Most of the day, they sat half idle, costing money for nothing. And every new city meant buying more, weeks in advance, guessing how much power they'd need before they needed it.

"We're always either short of machines or wasting them," Meera said. "Never just right. And buying them is like ordering an umbrella a month before the rain, hoping you guessed the storm."

Mohan had come over for this one. It felt like a sit-down-properly problem. He looked unusually serious, and she thought at first it was about the machines. It was not only about the machines.

"You're describing the old way of owning computers," he said. "You buy the machine. The data lives on it. The power lives on it. They're stuck together. So to keep your data, you keep the machine, running, day and night, whether you're using it or not." He drew the old way, one heavy box. "It's like buying a wedding hall

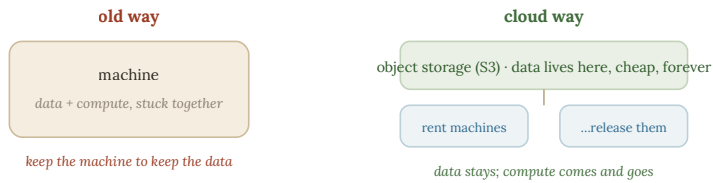
because you'll host a wedding once a year, then paying rent on it every single day it stands empty."

"That's exactly what it feels like. So what's the other way?"

"You rent. You use other people's computers, in enormous shared buildings, and you pay only for the minutes you actually use. That's all the **cloud** really is. Not a mysterious thing in the sky. Just renting computers by the minute, from someone who owns millions of them." He said it plainly. "You needed a wedding hall for one evening? You rent it for the evening, and walk away, and pay nothing tomorrow."

"Renting computers by the minute," Meera repeated. "So when Dev's big report runs, I rent the power for those minutes, then give it back."

"And here's the piece that makes it truly work," Mohan said. "You stop keeping data and power stuck together. You put your data somewhere cheap and permanent, where it just sits, safe, costing almost nothing. And separately, when you need to work on it, you rent power, point it at the data, do the job, and release the power." He drew the new way.




*Separating storage from compute. Data rests cheaply in object storage; machines are rented only when you compute, then released. The data never holds the machines hostage.*

"So the data stays still and cheap," Meera said, reading it. "And the power comes and goes as I need it. They're not stuck together anymore." She looked at the two shapes, the heavy old box and the light new pair. "This is the answer to everything I complained about. I never buy ahead. I never pay for idle machines. When Hyderabad launches, I just... rent more, for as long as the launch needs, then let it go."

"Separating the data from the power," Mohan said. "People call it separating storage from compute. It sounds dry. It's one of the biggest shifts in the whole trade. It's why a four-person company in Bengaluru can now do what only giant companies could do ten years ago." He looked at her. "It's why *you* can."

"The data sits cheap and forever. The power is rented by the minute, and pointed at the data when I need it." Meera sat back. "It's kinder to the money, and kinder to me. No more guessing the storm."

· · ·

They set up the plan to move Zaayka to rented machines. And then Mohan was quiet for a while, turning his empty chai glass in his hands, and Meera knew him well enough now to wait.

"I handed in my notice today," he said. "At the giant glass company. I'm leaving."

Meera put down her pen.

"You've been there six years," she said. "You built things for a company so big the reports are about other reports. You always said it was boring, but you never left. Why now?"

Mohan looked at the glass in his hands for a moment.

"Because I've spent a year building the most interesting things I've built in my life," he said. "And none of them were in that build-

ing. They were here. On this floor. Over your terrible chai. On a terrace, on a phone at midnight, on the back of your notebooks." He set the glass down. "I kept going in to that grey tower and doing grey work, and then coming here and feeling awake. A person only puts up with that split for so long before he notices which half is real."

"So what will you do?"

"Smaller things. Truer things. Work that stays close to the people it's for." He paused. "Work that keeps me near the good problems. And near the people I'd rather solve them with."

Meera's heart did something complicated.

"Are you leaving the giant company for Zaayka?" she asked. Carefully. Because it was a large question, and only half of it was about the company.

Mohan met her eyes. And he chose his words the way you choose your footing on a narrow ledge, knowing exactly how far the drop is.

"I'm leaving it for something smaller and warmer than a glass tower," he said. "For work that matters to real people I can see. Sri. Lakshmi. Dev." A pause, one beat too long. "For someone whose problems I'd rather be solving than anyone else's."

He did not say *you*. He said *someone*. But the word landed in the room as clearly as if he had, and they both heard the shape of the name he had not said, sitting in the air between them like a valid-from date, waiting for its year.

"That's a good reason," Meera managed.

"I thought so," said Mohan, and looked at her a moment longer, and then, mercifully, reached for a biscuit, and let them both step back from the ledge, hearts pounding, onto the safe wide ground of the work.

WHAT MEERA LEARNED

The **cloud** is, plainly, renting computers by the minute instead of buying them, so you can summon a hundred machines for an hour and release them when the job's done. Its deepest idea is **separating storage from compute**: data rests cheaply and permanently in **object storage** (such as Amazon S3), and machines are rented only when you need to compute, then released. The data never holds the machines hostage.

How you store files matters. **Parquet** is the format built for this world: it stores data by column (so questions read only the columns they need) and compresses well (because similar values sit together), making files small and queries fast, far better than plain CSV for large data.

In a real job: "storage and compute are separate" underpins nearly every modern data platform, and "store it as Parquet, partitioned by date" is standard practice that quietly saves large amounts of money and time.

*Let the data rest cheaply and forever,*
*and rent the muscle only when you need to lift.*

CHAPTER EIGHTEEN

# Lakes & the Lakehouse

*Where the lake and the warehouse meet*

. . .

For the first time, Meera solved it before she called him.

The problem had arrived that morning. Zaayka was drowning in new kinds of data. Not just neat orders in tidy tables anymore. Now there were photos of dishes from the kitchens. Long text reviews from customers. Streams of taps and scrolls from the app. Some of it was neat and table-shaped. Much of it was not. And her lovely warehouse, built for tidy tables, had no idea what to do with a photograph.

A year ago she would have called Mohan at once. Today she made a chai, sat on the floor with her notebook, and thought. And by afternoon, she had drawn something. She did not know if it was right. But it was hers.

Then she called him, not to ask, but to show.

"I think we've outgrown the warehouse," she said. "It only wants tidy tables. But half our data now is messy. Photos. Reviews. Taps. So I want a second place. A big, cheap place that will hold *anything*, neat or messy, in whatever shape it arrives. And then I want to refine it, in stages, until the top layer is clean enough for Dev's reports."

There was a pause on the line.

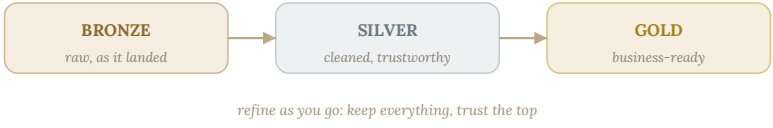
"Keep going," Mohan said. "You've clearly got the whole thing. I just want to hear you say it."

"So the messy holding place, I looked it up, it's called a **data lake**. It holds everything, raw, in any shape, cheaply. No rules at the door. Just, let it all in." She turned her drawing over, though he couldn't see it. "But a lake alone is a swamp. Everything dumped in, nothing trustworthy. So I don't just dump. I refine, in layers."

"Tell me the layers."

"Three. The bottom layer is the raw stuff, exactly as it landed, untouched. I keep it forever, because you taught me, never throw away the original. Then a middle layer, where I've cleaned it, fixed the spellings, tidied it, made it trustworthy. Then a top layer, shaped and ready, the clean tables Dev and Nisha actually use." She hesitated. "I don't have names for the layers. I just called them bottom, middle, top."

"The world calls them bronze, silver, and gold," Mohan said, and she could hear him smiling. "Raw bronze at the bottom. Cleaned silver in the middle. Polished gold on top. But your names were better. Refine as you go, keep everything, trust the top."




*The medallion layers. Raw data (bronze) is cleaned into trustworthy detail (silver), then shaped into business-ready tables (gold). Nothing is thrown away; trust increases as you climb.*

"Bronze, silver, gold," Meera said, writing it on her drawing. "Keep the raw bronze forever. Clean it into silver. Polish it into

gold. And anyone can always walk back down to the raw, if next year's question needs it."

"And there's a name for the whole clever thing you've just designed," Mohan said. "A lake that holds everything, cheap and open, but with warehouse-quality tidy tables built right on top of it, so you get both, the openness of a lake and the trust of a warehouse. People call it a **lakehouse**. Lake plus warehouse. Both, in one."

"A lakehouse," Meera said. "Of course it has a name. Everything good has a name." She looked at her drawing, the drawing she'd made alone. "I built the shape before I knew the word for it."

"You did. Which is the right order, honestly. The word is just a handle. You made the thing."

There was a silence then, and it was a full one.

"You didn't need me today," Mohan said. He did not say it sadly. He said it the way you say a thing you have been waiting a long time to be true.

"I wanted you, though," Meera said, before she could decide whether to. "That's different from needing. I solved it alone. And the whole time, the only thing I wanted was to show it to you. Not to be rescued. Just... to be seen doing it."

Another silence. She could hear him breathing, choosing.

"Meera." His voice had changed. "For a year I've called myself your teacher. It was a comfortable word. It let me sit close to you and pretend the sitting close was about the work." A pause. "But you just designed a lakehouse on your kitchen floor without me. So I can't really call myself the teacher anymore, can I. That word's used up."

"Then what are you?" Meera asked. Her heart was very loud.

"I don't know yet," he said. "But I know I was never *only* the teacher. Even at the start. I think we both knew that. I just had the word to hide behind, and now I don't."

Meera sat on her floor, in the last gold light, with a drawing she'd made alone and a phone warm against her ear, and understood that a door which had been held shut for a year had just moved on its hinge. Not opened. But moved. Enough to feel the air change.

"We don't have to name it tonight," she said softly.

"No," he agreed. "We're good at keeping the raw, you and I. Keeping the true thing, unshaped, until we're ready to refine it." A smile came back into his voice, gentler than his usual one. "Bronze, for now. We'll get to gold."

And they left it there, in the bronze, raw and kept and true, the most honest layer, the one everything else is one day built from.

WHAT MEERA LEARNED

A **data lake** keeps all your data, raw and in its natural state, cheaply in object storage. It throws nothing away, but without discipline it becomes an untrustworthy swamp. The fix is to refine data in layers, nicknamed **bronze** (raw, as it landed), **silver** (cleaned and trustworthy), and **gold** (business-ready), the **medallion architecture**.

Open table formats like **Iceberg** and **Delta** sit on top of the lake's files and give them warehouse manners back: safe row updates, transactions, protection from concurrent corruption, even viewing data as it was in the past. A lake with these is a **lakehouse**: the openness and low cost of a lake, with the safety and structure of a warehouse.

In a real job: "bronze/silver/gold" and "we're on Iceberg" (or Delta) are everyday phrases. The lakehouse is where a large share of modern data platforms now sit.

*Keep everything like a lake, trust it like a warehouse, and refine as you climb from raw to gold.*

PART FOUR

# Data That Flows

*Until now, Zaayka's data has been a thing you store and then study. But customers want to watch their food coming, live, on a moving map. Data stops being a lake you visit and becomes a river that never stops. This changes everything, and asks the most of Meera yet, just as the thing between her and Mohan grows too large to keep calling friendship.*

· · ·

CHAPTER NINETEEN

# Streaming & Kafka

*The map that moves*

· · ·

Customers wanted to watch their food coming, and that small wish broke everything Meera had built.

It sounded simple. A little map, in the app, showing your rider moving toward you. A dot, sliding along the streets, getting closer. Every other food app had one. Zaayka's customers had started to ask why theirs didn't.

But everything Meera had built was designed to gather data into piles and study it later. Once a night, the changes travelled. Once a night, the reports refreshed. That was fine for revenue and favourites. It was useless for a rider moving *right now*. A map that updates once a night is not a map. It is a history lesson.

"Everything I've built waits," she told Mohan. "It collects, then it thinks. But a rider on a map can't wait. The dot has to move the instant the rider moves. I don't know how to build a thing that never stops to think."

"You remember, a long time ago, I kept leaving a door half open," Mohan said. "A faster way. A map that moves. Not yet, I kept saying." He smiled down the phone. "It's yet. Come on. This is the door."

"Everything you've built so far is *batch*," he said. "Gather a pile, handle the pile, rest. Gather, handle, rest. It's calm, and it's power-

ful, and for most things it's exactly right." He drew the old rhythm. "But a moving rider isn't a pile. It's a stream. A rider's phone says *I'm here*, then a second later *now I'm here*, then *here*, on and on, a hundred little messages a minute, never stopping. You can't gather that into a nightly pile. You have to handle each message the instant it lands."

"Handling data the moment it arrives, one event at a time, forever," Meera said. "Instead of gathering it into piles."

"That's the whole shift. It's called **streaming**. Batch rests between piles. A stream never rests. Each little event, *I'm here*, flows in and gets handled and flows out to the map, live, endlessly." He looked at her drawing. "Now, the clever part is the shape in the middle. Because a rider's *I'm here* isn't wanted by just one thing. The map wants it. The delivery report wants it. The customer's phone wants it. So you don't send it straight anywhere. You write it into a log."

"A log?"

"Picture a long strip of paper. Every event that happens anywhere in Zaayka gets written on the next line, in order, the moment it happens. Rider moved. Order placed. Rider moved again. It never gets erased. It just grows, a perfect ordered diary of everything, as it happens." He drew it.


*Kafka is a log: an ordered, append-only list of events. Producers (rider phones, the app) write events onto it; consumers (the live map, reports) each read along at their own pace.*

"And then anyone who cares can read the strip," Meera said slowly, seeing it, "at their own speed. The live map reads it to move the dots. The report reads the same strip to count deliveries. They don't disturb each other. They're all just reading the same diary, each at their own pace." She looked up. "The ones writing events, and the ones reading them, never have to know about each other. They only share the strip."

"You've got it exactly. The writers are called producers, the readers consumers, but the words don't matter. What matters is the shape. One ordered log in the middle. Many things writing to it. Many things reading from it. Nobody waiting on anybody." He sat back. "That log is the spine of every live system you've ever used. The map, the messages, the little *your order is being prepared*. All of it, one growing strip, read live."

"A river, not a lake," Meera said. "You don't visit it. You stand in it, and it never stops."

· · ·

They built the first version that week. And the night it went live, Meera sat in her flat and watched the test map, and something happened she had not expected.

Mohan had gone out on a scooter himself, to test it from the road, playing rider. And so the single dot moving across the little map on her screen, sliding street by street through the dark city toward her building, was him. His actual position, live, each *I'm here* flowing through the log she'd built and out onto her screen as a moving point of light.

She should have been watching for bugs. Instead she watched the dot. She watched it turn onto the main road. She watched it pause at the signal by the temple. She watched it come, street by street, closer, and she understood that she had built, without meaning to, the exact thing her heart had been doing for a year: tracking one particular dot through the whole moving city, and caring only about that one.

Her phone buzzed. A message from the dot. *your streaming works. i can see you've been watching my little light come the whole way. draw whatever conclusions you like.*

Meera looked at the map. One dot, almost at her door now. She did not draw any conclusions. She just went down to open it, before he could knock the two soft raps and one, because tonight, for once, she did not want to wait even that long.

WHAT MEERA LEARNED

**Batch** gathers data into piles handled on a schedule; **streaming** handles data as a never-ending flow, for things that can't wait, like a rider's live location. In streaming you stop thinking in still tables and start thinking in **events**: little time-stamped messages announcing that something just happened, arriving one after another.

**Kafka** is the standard tool for this, and underneath it is simply a **log**: an ordered, append-only list of events. Producers write events onto it; consumers read along at their own pace. Its power is **decoupling**: writers don't know readers, so you can add new consumers (a live map, an alert, a dashboard) to an already-flowing stream without disturbing anything.

In a real job: Kafka (and the event-log idea) sits at the heart of countless real-time systems. "Put it on the event stream" is how modern systems stay flexible and live.

*Some data can't wait to be gathered into piles.*
*Let it flow as events, and let many things listen.*

CHAPTER TWENTY

# Delivery Guarantees

*Exactly once, please*

. . .

The river Meera had built turned out to be a liar, and she found out during dinner rush.

The live map worked beautifully, most of the time. But now and then a rider's dot would jump backward, or freeze, or a delivery would show as arrived before it left. The stream was not the clean, orderly diary she had pictured. It was messy. Messages arrived late. Some arrived twice. Some, on a bad network, arrived out of order, a rider's *I'm here* from thirty seconds ago landing after a newer one.

"The stream lies," she told Mohan, frustrated. "I thought events would arrive in order, once each, instantly. They don't. They come late, they come doubled, they come backward. A rider goes through a tunnel and his phone saves up five *I'm here* messages and dumps them all at once when he comes out, out of order." She rubbed her eyes. "How do I build anything true on a stream that arrives like that?"

"Now you've met the real problem of streaming," Mohan said. "And it's the honest one nobody mentions at the start. The world is messy, and a live stream shows you every bit of the mess. So you stop hoping for a perfect stream. You choose, on purpose, which honest promise you can keep."

"There are three promises a stream can make," he said, "and you can only pick one. So pick with your eyes open." He drew them out.

### THE THREE HONEST PROMISES A STREAM CAN MAKE

| guarantee | meaning | the catch |
|---|---|---|
| **at-most-once** | never a duplicate | but you might lose events |
| **at-least-once** | never lose an event | but you might get duplicates |
| **exactly-once** | no loss, no duplicates | the dream, but genuinely hard and costly |

*Most real systems choose **at-least-once** (never lose anything) and then make handling **idempotent**, so the possible duplicates do no harm. That combination gives you exactly-once results without the full cost.*

"The first promise: never show a duplicate, but you might lose one now and then. The second: never lose one, but you might show a duplicate. The third: never lose, never duplicate, perfect, but genuinely hard and expensive, close to a dream." He looked at her. "For a rider's position, which would you choose?"

Meera thought about it properly.

"Never lose one," she said. "I'd rather get a duplicate *I'm here* than lose the rider and freeze the dot. A doubled message I can deal with. A lost one leaves the customer staring at a frozen map." She nodded. "The second promise. Never lose. Accept duplicates."

"That's what almost everyone chooses. Never lose anything. And then you solve the duplicates a clever way." He wrote a word. "You make the handling *idempotent*. It's a big word for a simple, beautiful idea. It means: doing the same thing twice has the same effect as doing it once."

"Say more. How can twice equal once?"

"Think of a light switch that isn't on-off, but a button that only ever says *be on*. Press it once, the light is on. Press it again, the light is still on. Press it five times, still just on. The extra presses change nothing." He looked at her. "So if a rider's *I'm at this corner* arrives twice, you set the dot to that corner, twice. Same corner both times. The duplicate does no harm, because setting a position to the same place twice is the same as once. You've made duplicates harmless instead of trying to prevent them."

"Idempotent," Meera said. "Twice is the same as once. So I stop fearing duplicates. I just make sure they can't hurt anything." She wrote it down. "Never lose, and make the doubles harmless. That's the whole trick."

"There's one thing left," she said. "The out-of-order problem. The tunnel. If I'm counting how many riders were out at eight o'clock, and some of the eight o'clock messages don't arrive until five past, when do I decide eight o'clock is *done*? If I decide too early, I miss the stragglers still coming out of the tunnel. If I wait forever, I never get an answer."

Mohan was quiet a moment, and something in his face softened, as if the question had reached past the machines.

"That is the deepest question in streaming," he said. "And the answer is a thing I find almost tender, for a piece of engineering." He wrote a word. *Watermark.* "You draw a line, on purpose, a little way behind the present. You say: I will wait this long for stragglers. Anything later than this, I accept I may have missed. The line is called a **watermark**."

"A watermark. A line that waits."

"A line that waits. Not forever, because forever gives no answer. But long enough. You choose how long to hold the door open for the ones still in the tunnel. Too short, and you abandon the stragglers. Too long, and life never moves on." He looked at her, and did

not look away. "It's a choice about patience. How long is it worth waiting, for the thing that might still be coming? You can't wait forever. But the good ones, the ones worth having, you wait a little longer for."

The lamp hummed. Meera looked at the word on the page. *Watermark.* A line that decides how long you're willing to wait for something to arrive.

"Are we still talking about streams?" she asked. It was becoming their oldest, truest joke, and their oldest, truest question.

Mohan set the pen down.

"You know we're not," he said. Plainly. No deflection this time. No biscuit. "We haven't been, for a long time. I think you know exactly how long I've been holding my watermark open." He held her eyes. "The only thing I don't know is whether I've waited too long, or not quite long enough."

Meera's heart was going very fast. This was the closest they had ever come, the door not just moved now but truly ajar, light spilling through.

"Not too long," she said, barely above the lamp's hum. "Not too long, Mohan."

He breathed out, slowly, like a man who has been holding something for a year.

"Then I can wait a little longer still," he said. "For the rest of it. Until it's fully arrived, and nothing's still in the tunnel, and we can be sure." A small smile. "You taught me the value of not rushing the last stragglers. I can hold the line open a while more. This one's worth it."

And he did not say the rest, and she did not ask him to, because the watermark was drawn now, gently, between them, and they both knew the thing was coming down the line toward them, late but not lost, and worth every held minute of the wait.

WHAT MEERA LEARNED

Real event streams are messy: messages can arrive twice, out of order, or late. Two ideas tame this. First, make event handling **idempotent**, doing something twice has the same effect as once (prefer "set this order delivered" over "add one to a counter"), so duplicates do no harm.

Second, know the three **delivery guarantees**: **at-most-once** (never duplicate, may lose), **at-least-once** (never lose, may duplicate), and **exactly-once** (neither, but genuinely difficult and costly). Most systems wisely choose at-least-once plus idempotent handling, achieving exactly-once results at far less cost.

For late and out-of-order data, a **watermark** is the considered line past which you stop waiting for stragglers and compute your answer, balancing timeliness against completeness.

In a real job: "make it idempotent" is among the most valuable habits in streaming and pipelines alike. Perfect delivery is rarely worth its cost; safe handling of imperfect delivery is the real skill.

*You cannot stop the river from repeating itself, so make sure hearing a thing twice does no harm.*

CHAPTER TWENTY-ONE

# Orchestration & Airflow

*The pipeline that looks after itself*

. . .

The nightly work had grown into a monster with too many moving parts, and one broken piece could poison everything.

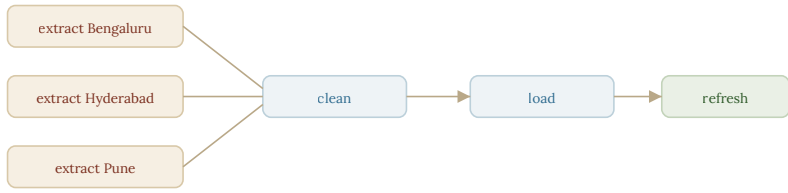
Every night now, a long chain of jobs ran while the cities slept. Pull the day's orders from Bengaluru, Hyderabad, and Pune. Clean them. Load them into the lakehouse. Refresh Dev's reports. Rebuild the favourites. A dozen jobs, each depending on the ones before it. And they were held together with a mess of alarms and hope.

When it worked, it was invisible. When one job failed, at three in the morning, the jobs after it ran anyway, on half-missing data, and everyone woke to reports full of quiet lies. Meera was tired of waking at three to a buzzing phone.

"It's too tangled to hold in my head," she told Mohan. "A dozen jobs, and each one depends on others, and if one breaks, the rest run on broken data and nobody knows until morning. I need something that understands the whole chain. What waits for what. What must never run if the thing before it failed."

"You need to stop being the thing that holds the chain together," Mohan said. "You need to draw the chain, once, clearly, and let a tool run it for you. And the drawing you'll make has a lovely shape. Let me show you."

"Write down your night's work as a set of tasks," he said, "with arrows for what waits for what. Extracting the three cities can happen at once, they don't depend on each other. But cleaning waits for all three to finish. Loading waits for cleaning. Refreshing waits for loading." He drew it, boxes and arrows.




*The night's work as a DAG. The three cities extract in parallel, then cleaning, loading, and refreshing follow in order. An orchestrator runs this automatically, every night.*

"See the shape," he said. "Every arrow points forward. Nothing ever loops back on itself. Cleaning never waits for something that waits for cleaning. It's a chain that only flows one way, from start to finish."

"A picture of what waits for what," Meera said, tracing the arrows. "The three cities pull at once, then everything funnels forward. Clean, load, refresh. And nothing points backward."

"That shape, tasks with arrows, always flowing forward, never looping, has a name," Mohan said. "It's called a **DAG**. Forget the letters. Just hold the picture: a map of jobs, and what each one waits for, all flowing one way." He looked at her. "And once you've drawn it, a tool can run it for you. It runs each job only when the jobs it waits for have truly finished. And if one fails, it stops everything downstream, so nothing ever runs on broken data again."

"So loading simply cannot run if cleaning failed," Meera said. "The tool just won't let it. No more three a.m. reports full of lies." She sat back, relieved already. "And it runs the independent things at once, and waits for the rest, exactly as the arrows say."

"Running your DAG for you, in the right order, waiting and retrying and stopping when it should, is called **orchestration**," Mohan said. "Like a conductor with an orchestra. Every player comes in at the right moment, and if one falls silent, the conductor knows, and doesn't just plough on." He smiled. "You stop being the conductor who never sleeps. You write the score, once, and hand it over."

"Write the score once. Let the conductor run the night." Meera looked at her DAG. "I could sleep through the night again."

That should have been the happy ending of the evening. It was not, because Nisha called with news.

The Hyderabad launch had gone so well that investors wanted Zaayka to open ten cities, fast. And they wanted Meera, the one who understood the data, to go to Hyderabad and set up the whole data operation there, in person, for three weeks. Leaving in two days.

Meera hung up and sat very still.

"Three weeks," she said. "In Hyderabad. Starting Thursday."

Mohan had gone quiet in the way he did when something had struck him hard. She was looking at his face, so she saw it: the small flinch, quickly hidden, of a man doing arithmetic he did not like.

"That's wonderful," he said. "It's exactly what you've earned. You should go."

"I know I should go."

"Then go." He said it gently. But his hands had gone still on the low table, and neither of them was looking at the DAG anymore.

And here was the cruelty of the timing, and they both felt it without saying it. For a year they had drawn slowly toward each other, one held look at a time. Last week, over the watermark, the door had finally come ajar. And now, before either had walked through it, three weeks and five hundred kilometres were about to open up between them, right at the moment the waiting had become almost unbearable.

· · ·

He left earlier than usual that night. At the door, he stopped, and for a moment she thought he would finally say it, the whole thing, the unsaid name, the rest of the sentence he had left open at the watermark.

He didn't. He looked at her for a long moment instead, memorising her a little, the way you do before a distance.

"Go and be brilliant in Hyderabad," he said. "Build them a beautiful thing. Come back."

"That's all? Come back?"

"That's all. For now." A pause, and then the closest he came: "The stragglers can wait three more weeks, Meera. I've held the line this long. I'm not going to abandon it two days before it arrives." He smiled, and it did not quite reach the ache in his eyes. "When you're back. There's a thing I've been not-saying for a year. When you're back, I'll say it. Thursday, you leave. Some Thursday soon, you'll return, and I'll be here, and I'll say it."

"Promise?"

"I promise. Some things you wait a little longer for, and they're worth it. You taught me that." Two soft steps, and a wave, and he was gone down the stairs, and Meera stood at the top holding the

word *Thursday* like a watermark of her own, a line drawn in time, a date to wait for.

And the flat had never felt so quiet, or the three weeks ahead so long, or the thing still coming down the line toward them so late, so certain, and so nearly, nearly here.

WHAT MEERA LEARNED

When your data work becomes a fixed procedure, run these steps in this order, retry failures, do it every night, you should hand it to a machine, not do it by hand. This is **orchestration**. You describe the work as a **DAG** (a directed acyclic graph): boxes for tasks, arrows for "must finish before," no loops. Independent tasks can run in parallel; dependent ones wait.

An orchestrator like **Airflow** runs the DAG on a schedule and, crucially, adds reliability: it retries failed steps automatically, alerts you only when something truly needs you, keeps a history of every run, and can **backfill** past days to fill gaps. It turns a fragile nightly ritual into infrastructure you can sleep through.

In a real job: orchestration is the backbone of data platforms. If you're manually running steps in order, that's a sign it's time for a DAG.

*If you have become the thing that runs the steps, write down the steps, and go to sleep.*

PART FIVE

# A Grown-Up System

*Zaayka is no longer four friends and a spreadsheet. It has people, investors, responsibilities, a system a whole company leans on. The last things Meera learns are the ones that turn a clever build into something that can be trusted, and outlast its makers. And the thing between her and Mohan, so long unspoken, finally, quietly, learns its own name.*

· · ·

CHAPTER TWENTY-TWO

# dbt & Analytics Engineering

*Transformations you can trust*

. . .

Meera came home on a Thursday, three weeks older, and Mohan was waiting at the airport.

She had not asked him to come. But there he was, past the baggage belts, in a shirt she liked, holding a paper cup of the terrible airport chai as a joke, and the sight of him undid three weeks of careful composure in about a second.

They did not say the thing at the airport. It was too loud, too bright, too full of strangers. But he took her heavy bag without asking, and their hands managed the handoff, and this time neither of them pretended the other's hand wasn't there. They just walked out into the Bengaluru evening, and it was gold, and everything felt very close to arriving.

"There's a work problem," he said, on the drive. "A real one. Kavya found it."

"Who's Kavya?"

"Your new analyst. Sharp. Started while you were away. She found something you'll want to see tonight, not tomorrow." He glanced at her. "And then, after, there's the other thing. The thing I said I'd say on a Thursday." A pause. "It's Thursday."

Meera's heart turned over. "It's Thursday," she agreed.

At the office, Kavya's problem was waiting, and it was a good one.

Kavya was young and quick and a little nervous to meet Meera, whose warehouse she had been living inside for three weeks. And she had found a mess. Everyone on the growing team had started writing their own definition of simple things. What counted as a *delivered* order. What counted as *revenue*. Dev's report said one thing. Kavya's said another. Nisha's said a third. Same word, three meanings. Sri's five spellings, come again, grown up and wearing a suit.

"It's the oldest problem in the book," Meera said, and almost laughed, because it was. "One thing, defined five ways. I built this whole company to kill that, and it's crept back in at the top."

"So kill it again, the same way," Mohan said. "One definition, in one place, that everyone builds on. Kavya, show her what you tried."

Kavya, gaining courage, pulled it up. "I started writing each definition once, as its own step. Like, here is *delivered orders*, defined one way, forever. And then *revenue* builds on top of *delivered orders*. And *revenue by kitchen* builds on top of *revenue*. Each thing defined once, and everything else refers back to it, instead of redefining it."

Meera looked at what Kavya had built, and felt a deep, familiar satisfaction.


*one shared definition of "delivered", built on once, reused everywhere*

*dbt models build on each other. "Delivered orders" is defined once; every revenue table builds on that single definition, so nobody's numbers can quietly disagree.*

"It's the whole book, again, at a higher floor," Meera said softly. "One fact, one home. Define *delivered* once. Build *revenue* on it. Build the reports on *that*. A chain of definitions, each one built on the trusted one below, none of them ever repeated." She turned to Kavya. "This is exactly right. There's a tool that does this beautifully, so the whole team writes definitions this way, each built on the last, tested, in one shared place. It's called **dbt**. But you invented the shape yourself before you knew its name. That's the only way that really teaches you."

Kavya glowed. And Meera realised, watching her, that she had just done for Kavya what Mohan had done for her, on a floor, over chai, a lifetime ago. The teaching, handed on. The chain, one more link.

· · ·

Kavya went home. The office emptied. And then it was just the two of them, and the gold gone to dusk in the windows, and the thing that had waited a year and three weeks, waiting no longer.

"So," Mohan said. He put down his pen. His hands, she noticed, were not quite steady. "It's Thursday."

"It's Thursday," Meera said.

He came around the low table and sat, not across from her as he had a thousand times, but beside her, close, the way you sit when the distance itself is the thing you can no longer bear.

"I've been organising my whole life around you for six years," he said. "Since college. Quietly. In the dark. Before either of us knew to ask the question. I built a career, and picked a city, and left a giant company, and I told myself each one was about the work." He looked at her. "It was never only about the work. It was about being where you were. I was doing the organising, long before the asking. You taught me the words for it yourself, without knowing you were describing us."

Meera's eyes were bright. "The work is in the organising," she said. "Done before the asking."

"Done before the asking. I organised my whole heart around you before I ever had the courage to ask if you felt it too." He took her hand, properly this time, the way a customer and a kitchen finally sit in the same honest row. "So I'm asking. On a Thursday, like I promised. I'm in love with you, Meera. I have been for longer than I've admitted, even to myself. That's the thing I've been not-saying for a year."

The words she had waited for, plain at last, no metaphor, no deniable joke, no *mostly*. Just the thing itself, said clearly, in the dusk.

"I kept the raw," Meera said, half laughing, half crying. "All this time. You told me to, on the very first pipeline. Keep the original, don't throw it away for the tidy copy. I kept the raw version of how I felt about you for a whole year, in a college T-shirt I never gave back, because I wasn't ready to refine it into words." She turned

her hand over in his. "I'm ready now. I love you too. I have for so long I can't find the start of it."

He let out a breath that had, she thought, been held since a rainy evening a year ago. And then he smiled, the real one, the one she had worked for on purpose for a year, and it was entirely for her.

· · ·

They did not do anything dramatic. They were not dramatic people. They sat close in the emptied office, in the last of the light, hands folded together on a low table covered in the diagrams of the thing they had built, and they talked, quietly, the way they always had, except that now nothing had to be deniable, and no look had to end one beat early, and no sentence had to be left open.

Outside, a temple bell rang somewhere down the road. Three slow notes, spaced apart, the way it had rung on the very first evening, a lifetime and a whole education ago.

"Are we still talking about data?" Meera asked, one last time, into his shoulder.

"No," Mohan said, and she could hear the smile in it. "No, we're finally not. We never really were. It was the coat the thing wore, until we were brave enough to see what was underneath."

And the bell finished its three unhurried notes, and the dusk came down soft over the city they had wired together, and the oldest question either of them had was, at last, quietly and completely, answered.

WHAT MEERA LEARNED

As a data team grows, people writing their own private SQL leads to numbers that quietly disagree (two definitions of "revenue"). **Analytics engineering** fixes this by treating SQL transformations like real software: shared, versioned, tested, and documented, instead of scribbles in private query tabs.

The tool that made this normal is **dbt**. You write each transformation as a **model** (named SQL producing one clean table), and models build on each other, so a definition like "delivered orders" is written once and reused everywhere. You add **tests** (this is never null, this is unique, revenue is never negative) that run with the models and catch errors before anyone sees them. The result is one shared, trustworthy set of numbers.

In a real job: dbt and analytics engineering are now central to modern data teams. The real value isn't fancy SQL; it's a team agreeing what its metrics mean and writing that agreement down where it can't drift.

*Don't let everyone keep their own private truth.*
*Write the meaning down once, test it, and share it.*

CHAPTER TWENTY-THREE

# Data Quality & Observability

*Keeping data honest*

. . .

The worst kind of data problem is the one that doesn't break anything.

It was a Tuesday, a few weeks into the new life, the one where Mohan held her hand across the low table now and nobody at the office pretended to be surprised. And on that Tuesday, the reports were wrong, and nothing had crashed.

That was what frightened Meera. Nothing had failed. No alarm. No error. The pipelines ran green all night. But the day's revenue was quietly, badly wrong, and Nisha had nearly sent bad numbers to an investor before Kavya caught it by luck.

"A crash I can handle," Meera told Mohan. "A crash screams. This just... smiled and lied. Everything ran. Everything was green. And the numbers were garbage. If Kavya hadn't happened to glance at it, we'd have sent it out." She shook her head. "How do I catch a problem that doesn't make any noise?"

"You stop waiting for noise," Mohan said. "A crash announces itself. Bad data just sits there, looking exactly like good data. So you don't wait to be told. You go and check, on purpose, every single day. You give the data a health check before you trust it."

"Think like a doctor," he said. "A few simple checks, run every morning, that catch most illnesses before they spread." He drew them out, one plain question each.

### THE EVERYDAY DATA-QUALITY CHECKS

| check | asks | would have caught Tuesday? |
|---|---|---|
| **freshness** | is the data recent enough? | no (data was fresh, just wrong) |
| **volume** | is the row count in the normal range? | maybe (fewer complete rows) |
| **nulls** | are required fields present? | yes, missing line totals |
| **uniqueness** | are keys unique, no duplicates? | not this time |
| **ranges** | are values within sane bounds? | yes, revenue oddly low |

*A **null check** on line totals would have caught Tuesday the moment the broken data arrived, before it ever reached a dashboard.*

"Is the data recent enough, or is it stale? Is the amount about what you'd expect, not suddenly half, not suddenly double? Are the fields that must be filled actually filled? Are the keys still unique, no sneaky duplicates?" He looked at her. "Simple questions. But run every day, automatically, before anyone trusts a number, they catch the quiet lies while they're still small."

"So I test the data, not just the code," Meera said slowly. "I always tested whether the pipeline *ran*. I never tested whether what came out was *true*." She looked at the checks. "This is checking the milk before you drink it. Not just checking the fridge turned on."

"Checking the milk, not the fridge. That's better than the textbook again." Mohan smiled. "This whole practice has a name, **data quality**, and these little standing checks are called data tests. But

the idea is older than any tool. Don't trust. Verify. Every day. Especially when nothing looks wrong. *Especially* then."

"Because the quiet ones are the dangerous ones," Meera said. "The loud failures announce themselves. The quiet ones you have to go looking for."

"The quiet ones you have to go looking for. Every single day. Forever." He looked at her a moment longer than the sentence needed. "It's not glamorous work. It's just care, repeated. Turning up every day to check on a thing you're responsible for, whether or not it's asking for help."

· · ·

They built the morning checks together, and Meera set them to run before dawn, so the data would be checked and trustworthy before anyone woke to use it.

And walking home that night, hand in hand through the warm streets, Meera thought about what he'd said. *Care, repeated. Turning up every day to check on a thing you're responsible for.*

Because that, she was learning, was what the love actually looked like, now that it had a name. It was not the grand thing from the airport, though that had been real. It was quieter, and it was daily. It was Mohan noticing she hadn't eaten and putting a plate in front of her without a word. It was her learning which silences of his needed company and which needed room. It was two people running a quiet health check on each other every single day, catching the small wrongs before they grew, especially on the days nothing looked wrong.

"You've gone quiet," Mohan said, swinging their joined hands a little.

"Just thinking that love is mostly data quality," she said. "The daily checks. The turning up. The catching of small things before they get big. Not the dramatic part. The Tuesday part."

Mohan laughed, delighted, the real laugh.

"That is the least romantic sentence anyone has ever said walking home holding hands," he said. "And it's the truest thing about love I've ever heard. Marry me eventually, you strange, wonderful woman."

"Eventually," Meera agreed, entirely happy, and they walked the rest of the way home in the kind of comfortable silence that, she had learned long ago on a terrace, was really its own quiet conversation, and had been all along.

WHAT MEERA LEARNED

The most dangerous data problems are not crashes but **wrong-but-plausible** numbers that hide behind a confident face. Catching them requires actively checking the data against what should be true, **data quality**, and continuously watching for silent problems, **observability**.

A handful of automatic checks catch most trouble: **freshness** (recent enough?), **volume** (normal row count?), **nulls** (required fields present?), **uniqueness** (no duplicate keys?), and **ranges** (values within sane bounds?). Run them the moment data arrives, so problems are caught at the source, cheap and contained, not at the end on a dashboard.

Where data passes between teams, a **data contract** is an explicit, checked promise about its shape and meaning, so a team that breaks it fails loudly on their own side, before the damage spreads downstream.

In a real job: data quality is where trust is won or lost. The best of this work is invisible, the disasters that quietly never happen, and that invisibility is exactly the point.

*A crash announces itself; a wrong number hides.*
*So check the data's honesty before anyone trusts it.*

CHAPTER TWENTY-FOUR

# Governance & Lineage

*Where did this number come from?*

. . .

An investor pointed at a single number on a slide and asked where it came from, and the room went silent.

It was a big meeting. Nisha was presenting Zaayka to people who might give it a great deal of money. On the screen was a number, the company's monthly revenue, clean and confident. And one investor, a careful woman with sharp eyes, tapped it and said: "That number. Walk me back through exactly where it comes from. Every step. I want to know it's real."

Nisha looked at Meera. And Meera realised she could not, on the spot, prove it. She believed the number. But she could not trace it, live, all the way back to the dosas it was made of.

She got them through the meeting. But that night she was quiet, and Mohan noticed, because noticing was what he did now.

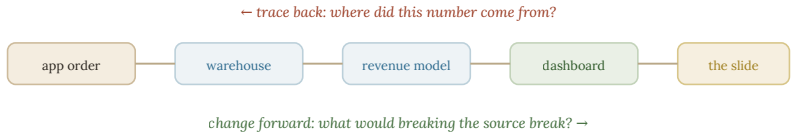
"I couldn't prove it," she told him. "The number was right. I know it was right. But she asked me to trace it back, step by step, to the real orders, and I couldn't do it fast enough to be convincing. A number you can't trace is just a rumour with good manners."

"A rumour with good manners," Mohan repeated. "That's exactly what an untraceable number is. And the fix is one of the quietly important things. You need to be able to follow any number, all the

way home." He pulled the laptop over. "Both directions, actually. Let me show you."

"Both directions?"

"Watch." He drew the chain. "The number on the slide came from the dashboard. The dashboard came from the revenue model. That came from the warehouse. That came from the app orders. That came from a real person buying a real dosa from Sri." He drew arrows all the way down. "So, backward: you can start at the slide and trace it, step by step, home to the dosa. That answers the investor. *Here is exactly where this came from.*"




*Lineage is the traceable path of a number through every step that made it, walkable backward (where did this come from?) and forward (what would a change break?).*

"And forward?" Meera asked.

"Forward is the other gift. Suppose Sri renames a dish, or Arjun changes something in the app. You can trace *forward* and see everything that number touches downstream. Every report, every dashboard, every slide that would shift if you changed the source. So you never break something by accident, because you can see, in advance, everything a change would ripple into."

"So backward answers *where did this come from*," Meera said, tracing it. "And forward answers *what would I break if I changed this*. The whole family tree of a number, both ways." She looked up. "If I'd had this today, I'd have walked that investor from the slide

down to Sri's dosa in ten seconds, and she'd have believed every rupee."

"That ability, to trace any number back to its source and forward to everything it feeds, is called **lineage**," Mohan said. "The family tree of every fact. Where it came from, and what it touches." He smiled. "You built a company to make data trustworthy. Lineage is how you *prove* it's trustworthy, to someone who wasn't there. It's trust you can show, not just trust you feel."

"Trust you can show," Meera said. "A number that can walk you home."

· · ·

They built the lineage view over the next week, and the next time an investor asked, Meera traced a number from a slide down to a single dosa at Sri's counter in front of the whole room, and Nisha later said it was the moment the deal was won.

But the thing Meera remembered from that chapter of her life was smaller, and happened at the office late one night, when Dev found her still working and sat down across from her.

Dev, the careful worrier. The one who had asked the very first question, a lifetime ago, that had sent her to the floor with a pencil. He looked at the lineage view on her screen, the whole company traced from slide to dosa, and then he looked at her, and there was something almost fatherly in it.

"You know," he said, "when we started, you were the marketing one. The one who was scared of the spreadsheet." He nodded at the screen. "Now the whole company's trust runs through you. Investors believe us because of what you built. I just wanted to say it out loud once. We'd be nowhere without you. I'd be nowhere

without you." He squeezed her arm, gruffly, the way Sri did, and got up before it could get sentimental. "Anyway. Go home. Mohan's waiting, and everyone can see it, and it's very sweet, and please stop pretending it's a secret."

Meera laughed, and her eyes stung a little. She had traced a lot of numbers home that week. But the one that reached her most was Dev's plain sentence, followed all the way back to a frightened woman on a floor, and forward to here: a leader the whole company leaned on, trusted, and quietly loved.

WHAT MEERA LEARNED

**Lineage** is the ability to trace any number back through every step that produced it, and forward to everything it feeds. Backward, it answers "where did this come from?" (priceless when a number is questioned); forward, it answers "if I change this, what breaks?" (priceless before you change a source).

Lineage is part of **governance**: the practice of being able to answer honest questions about your own data, what is it, where's it from, who owns it, who may see it, can we trust it. A **catalog** (a searchable directory of your data) lets people find and understand tables without asking around; clear **ownership** gives every important table a human who cares about it. Governance isn't paperwork; it's refusing to have data nobody understands or can trace.

In a real job: as a company grows, "where did this number come from?" becomes a daily question. Lineage and a data catalog are how mature teams answer it in minutes instead of panic.

*The error is rarely the disaster.*
*Not being able to trace the error is the disaster.*

CHAPTER TWENTY-FIVE

# Privacy & Security

*Looking after people's data*

. . .

It became real, and frightening, the day a stranger called the office asking for a customer's address.

He was smooth about it. He said he was a rider who'd lost a delivery note, and he needed the address for order such-and-such. He named a real order number. He almost got it. A new support hire nearly read the address aloud before something felt wrong and she put him on hold.

The order was Priya's. Customer 318. The very first customer Meera had ever traced, a lifetime ago, when Priya was just a name in a spreadsheet. Now Priya was a real woman in a flat in Malleshwaram, and a stranger had just tried to use Zaayka's data to find her.

Meera felt cold for the rest of the day. That night she was fierce about it in a way Mohan had not seen before.

"Everyone on my team can see everything," she said. "Full names. Phone numbers. Home addresses. Every one of them, all the time. And today a stranger nearly talked his way to Priya's front door through us." She looked at him. "I built all this to make the data useful. I never once stopped to ask who should be allowed to *see* it. That's not clever. That's dangerous."

"Now you've felt the thing that separates a real engineer from a clever one," Mohan said quietly. "Data about people isn't like data about orders. It carries a duty. And the duty is simple to say: people should be able to see exactly what they need to do their job, and not one thing more." He pulled the laptop over. "The good news is you already know the shape of the fix. You've been doing a version of it since the very first chapter."

"I have?"

"Since the first day. *Ask for less, get less.* Remember? You don't show every column. You show only the ones the question needs." He drew it. "Priya's order has two honest faces. The delivery driver needs her address, so the delivery view shows it, tightly guarded. But Dev, studying revenue, never needs her address or her name. He needs her area and her total. So his view hides the rest."

### THE SAME ORDER, TWO VIEWS

| | **full view · for delivery, tightly guarded** | **masked view · for analytics** |
|---|---|---|
| customer | Priya | customer 318 |
| phone | +91 98••• ••••• | (hidden) |
| address | flat in Malleshwaram | area: Malleshwaram |
| total | ₹370 | ₹370 |

*Analytics keeps everything useful (area, total, timing) and loses everything identifying. The revenue is just as true; Priya is just as safe.*

"The same order, two views," Meera said, reading it. "The full one, for delivery, locked down tight. And the masked one, for analytics, where Priya becomes customer 318 again, and her phone and her address just... aren't there. Dev can still study everything he needs. He just can't see who she is." She looked up. "He gets the shape of the data without the person inside it."

"The shape without the person. That's the whole art of it. Hiding the parts that identify someone, while keeping the parts you can safely study, is called **masking**. And the bigger rule, giving each person access to exactly what their job needs and nothing more, is the heart of data privacy." He looked at her. "Dev doesn't lose his revenue reports. Priya doesn't lose her safety. You stop choosing between useful and safe, and you get both, by showing each person only their honest slice."

"Only their honest slice," Meera said. "The driver sees the door. Dev sees the area. Nobody sees more of Priya than their job truly needs."

"And that stranger on the phone," Mohan said, "gets nothing, because there's no one left in the building who can see enough to be tricked."

. . .

They rebuilt the whole system of who-can-see-what over the next week. It was slow, unglamorous work, and Meera did every bit of it with a care that surprised even her.

Because it had stopped being about data. On the first day, a year and a lifetime ago, the data had been chits in a drawer, a puzzle to solve. Now, at the far end of every row, there was a person. Priya, in her flat. Sri, at his counter. Lakshmi, counting by hand. Every number traced home to someone real, someone who had trusted Zaayka with a piece of their life.

"I used to think the job was making the data useful," Meera said, on the terrace, late, Mohan's arm around her now in the way that was ordinary now and still not ordinary at all. "Then I thought it was making it true. Now I think it's making it *safe*. Looking after the people at the far end of all these rows."

"That's the whole arc of it," Mohan said. "Useful, then true, then safe. You've walked the whole road." He was quiet a moment, looking out over the city, all its lit windows, each one a person, each person maybe a customer, each customer a guarded row in a system she had built. "You started out protecting a spreadsheet from five spellings. Now you're protecting half a city from strangers on the phone. It's the same instinct, grown all the way up. Look after the thing you're responsible for."

Meera looked out at the lit city with him, and felt the love she had for this one man quietly widen, until it took in the whole glittering grid of them, every window a Priya, every Priya someone worth guarding. She had come to data to organise a drawer. She had stayed, she understood now, because organising, done with care, was just another word for looking after people. All of them. The ones you loved, and the ones you would never meet, safe in the rows of a thing you built well.

WHAT MEERA LEARNED

Data about people carries a duty that data about orders does not. **Personal data** (or **PII**), names, phone numbers, addresses, anything identifying a real person, is governed by laws (like GDPR) and, more importantly, by decency: collect only what you need, use it only as people agreed, let them see and delete it, and protect it.

The practical tools are **access control** (each person sees only what their job requires; a rider sees one customer's address only while delivering), **masking** (analytics replaces names and numbers with harmless stand-ins like "customer 318," keeping all the usefulness and none of the exposure), and **security** (encrypt the personal data that must exist, log who accesses it, and keep as little as possible for as long as necessary).

In a real job: handling PII responsibly is now a core, legally serious part of data engineering. Masked analytics views and least-privilege access are standard, and the best practitioners treat it as a matter of care, not just compliance.

*Behind every row about a person is a person. Keep only what you must, and guard it as if it were your own.*

CHAPTER TWENTY-SIX

# Infrastructure as Code

*Writing down the plumbing*

. . .

A single forgotten setting, made by hand months ago, took the whole company down for a morning.

It was nobody's fault, exactly. Long ago, setting up the machines, someone had clicked a setting, adjusted a permission, installed a thing, and moved on. It worked. Everyone forgot. Then one day a machine died and had to be rebuilt, and no one could remember exactly how it had been set up. They rebuilt it from memory, got one setting wrong, and Zaayka was dark for three hours while they hunted for the difference.

"We built our whole foundation by hand," Meera said, "clicking things, and then forgetting what we clicked. So when a machine dies, we're rebuilding it from memory, like trying to redraw a map we only saw once." She was frustrated. "There has to be a way to not rely on memory for the thing the entire company stands on."

Mohan smiled, and it was a particular smile, one she had come to know. It was the smile he wore when the answer was going to be an old friend.

"There is," he said. "And you're going to laugh, because you already know it. You've known it since the very first evening we ever spent together."

"Right now your infrastructure lives in memory and old clicks," he said. "The warehouse, the storage, the servers, the permissions, all set up by hand, once, and half forgotten." He drew the two ways. "So instead, you write it all down. Every piece. Not as notes. As a precise file that describes exactly what should exist. This warehouse. This storage bucket. This server. These permissions. Written out, exactly, in one place."


*Infrastructure as code. Instead of building machines by hand and forgetting how, you describe them in a file a tool can rebuild exactly, any time.*

"And then," Meera said, already seeing it, already smiling, "if a machine dies, I don't rebuild from memory. I just run the file again. And it builds the exact same thing. Every time. Identical. In minutes." She looked at him. "The setup stops living in someone's head. It lives in a file anyone can read."

"Describing your whole foundation as a written file, instead of building it by hand, is called **infrastructure as code**," Mohan said. "The whole company's bones, written down, precise, repeatable. A machine dies, you re-run the file, and an identical one takes its place in minutes. No memory. No guessing. No three-hour hunts for the one wrong setting." He looked at her, and his smile widened. "Now. Tell me what this reminds you of. Go back to the very beginning."

Meera thought. And then she laughed out loud, because there it was.

"It's the drawer," she said. "It's the very first thing you ever taught me. Don't keep it all jumbled in your head, or in a messy drawer where you can't find anything. Write it down. Give it a shape. Organise it, on purpose, before you need it, so that when the moment comes, the answer is just *there*." She shook her head in wonder. "The whole book, from the first evening, was one idea. And here it is again, at the very top, holding up the entire company."

"The work is in the organising," Mohan said softly. "Done before the asking. It was true of a drawer of chits. It's true of a company's whole foundation. It was true of everything in between." He took her hand. "You've spent two years learning thirty different-looking things. They were all the same thing, wearing thirty coats. Write it down. Give it a shape. Do the quiet work first."

. . .

They wrote the company's whole foundation into code over the following weeks, and never again lost a morning to a forgotten click. And on the evening they finished, Meera sat looking at the file, the entire bones of Zaayka, written down plainly where anyone could read them, and felt something come full circle.

Because there had been a time, at the very start, when she was the only one who understood how any of it worked. It had all lived in her head, precious and fragile, and secretly she had liked being the one who held it. It had made her necessary.

Now she had written it all down, in the open, where anyone could read it and run it and keep it alive without her.

"Doesn't it scare you?" Mohan asked, watching her face. "Writing it all down like this. You used to be the only one who knew.

Now it's all in the open. Anyone could keep it running. You've made yourself... less necessary."

"It used to scare me," Meera said. "When I was starting out, I held things close, so I'd be needed." She looked at the open file, the whole company legible on the screen. "But I don't want to be needed like that anymore. Held-hostage necessary. I want the thing I built to be so well-organised, so written-down, so *clear*, that it outlives me. That it doesn't need me at all." She turned to him. "The love isn't a secret anymore, and neither is the company. I wrote both of them down, in the open, plain. And it turns out that's the only kind of thing that lasts. The kind you're not afraid to let everyone see."

Mohan looked at her for a long moment, this woman who had once cried over a spreadsheet and now built things brave enough to outlast their maker.

"Written down, in the open, plain," he said. "It's a good way to build a company." He kissed her forehead, unhurried, entirely unhidden, in the lit office where anyone could see. "It's a better way to love someone."

WHAT MEERA LEARNED

Building servers and infrastructure by hand creates fragile one-offs that die with your memory of how you made them. **Infrastructure as code** fixes this: you describe what you want (a warehouse, a bucket, a server, permissions) in a file, and a tool like **Terraform** builds it, rebuildable identically in minutes, versioned and reviewable like any code.

Two companions share the same instinct. **Containers** package software with everything it needs, so it runs identically everywhere (no more "it worked on my machine"). **CI/CD** automates testing and deploying changes safely and the same way every time. Together they make a whole system repeatable, so nothing important lives only in one person's memory or on one precious machine.

In a real job: infrastructure as code (often Terraform), containers (often Docker), and CI/CD are standard practice. The principle is the same one that started the book: write it down, make it repeatable, don't keep anything important only in your head.

*A machine built by hand dies with your memory of it.*
*A machine written as code can always be built again.*

CHAPTER TWENTY-SEVEN

# Data for AI

*Feeding the machines*

. . .

For once, Meera was the one explaining, and Mohan was the one sitting on the floor being taught.

It had happened by accident, the way the best reversals do. Zaayka wanted to recommend dishes. If you loved Sri's masala dosa, what else might you love? Meera had spent a month deep in it, in the strange new country of machine learning, while Mohan was busy with the last weeks of his glass-tower job. And she had come back from that country knowing something he did not.

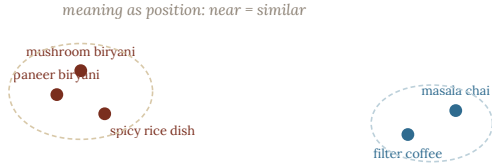
"Sit," she told him, taking the good pen from his hand, which made him laugh out loud. "Tonight I teach you. Try not to cry."

"I make no promises."

"A model can't learn from raw orders," she said. "I know that much now, because you drilled it into me. It needs history shaped into clean signals. How often a customer orders. What they re-order. Their favourite kitchen. Those shaped signals are called features, and making them, it turns out, is just... data engineering. My whole job. The thing you taught me. The machines can't learn until someone like me has shaped the food for them." She was enjoying this enormously. "So even the cleverest AI sits on top of the boring, holy work of organising data honestly. You were right. You're always right. It's insufferable."

"This is the best evening of my life," Mohan said. "Go on. Teach me the part I don't know."

"The part you don't know is how a machine holds *meaning*." She drew it for him, carefully, the way he had drawn ten thousand things for her. "You give every dish a position. Not on a map of the city. On a map of meaning. And you place them so that things which mean similar things sit near each other. The two biryanis land close together. The hot drinks cluster over here, coffee near chai. A thing's meaning becomes simply where it sits, and how near it is to everything else."




*Embeddings place things by meaning: biryanis cluster together, hot drinks cluster elsewhere. A vector store finds what's nearest in meaning, even with no shared words.*

"It's called an embedding," she said. "Meaning as nearness. Two things are alike if they sit close. And to recommend, you just look at what's near. You loved this dosa? Here's what sits closest to it in the map of meaning. That's the whole trick. Distance is similarity. Nearness is meaning."

Mohan was quiet, looking at the little map, at the biryanis sitting close, the hot drinks clustered warm together.

"Meaning as nearness," he said slowly. "Two things are what they are by how close they sit to each other." He looked up at her, and there was something in his face that had left the lesson en-

tirely. "That's the truest thing anyone's taught me in years. And you taught it to me."

"Are we still talking about embeddings?" Meera asked. It was the old joke, turned around now, handed back to him across two years.

"Mostly," he said, giving her back her own word, and they both smiled at the whole long history folded inside it.

They sat with it a while, the little map glowing, the two of them close on the floor, near in every sense the diagram had a word for.

"Can I tell you something," Mohan said, "that isn't about data. Or that is, but isn't." He turned the pen in his fingers. "I've been trying to work out, for years, how I know things. The real things. Not facts. The way you know the shape of a problem before you can prove it. And I think there are only three ways anyone ever learns anything that matters."

Meera went still, because his voice had changed into the low register, the one from before the true first note.

"There's seeing a thing truly," he said. "Really seeing it, the actual shape under the mess, the way you learned to see a kitchen where the spreadsheet saw a scribble. That's one. And there's doing the work, the patient organising, for its own sake, without grasping at the reward, the way you moved four hundred orders by hand at midnight because they deserved a true shape. That's two." He paused. "I always thought those were the only two. Thinking, and working. That's what I had. That's what I taught you."

"And the third?"

He looked at her for a long moment.

"The third one I didn't know," he said quietly, "until you. There's loving a thing enough to get it right. Not for the reward. Not even to understand it. Just because you love it, and love won't let you

leave it half-done or half-true." He set the pen down. "I could see, and I could work. You taught me the third one, Meera, without ever once meaning to. And it turns out to be the one that holds the other two together."

Meera could not speak for a moment. She had learned the whole craft from this man, two years of it, and here he was telling her that she had, without knowing, taught him the only part he had been missing.

"Seeing, and working, and loving," she said, when she could.

"Something like that." He smiled, and let it go lightly, the way he let all the largest things go lightly, and picked the pen back up. "Anyway. You were teaching me embeddings. Meaning as nearness. Show me the rest."

And she did. But she kept the three ways folded in her chest all evening, near and warm, like two biryanis on a map of meaning, sitting close because they meant the same thing.

WHAT MEERA LEARNED

AI and machine learning rest on data engineering. A model can't learn from raw data; it needs history shaped into **features**, clean, consistent signals (how often a customer orders, what they pair together), computed by a **feature pipeline**, which is the same ETL, quality, and orchestration you already know, pointed at feeding a model. The model is the last mile; the data engineering is the road, and a model is only as good as the data fed to it.

For finding things by meaning rather than exact match, each item is turned into an **embedding** (a list of numbers capturing its meaning, so similar things sit near each other), stored in a **vector store** that answers "what's nearest in meaning?" This powers modern recommendation and language-based AI.

In a real job: the rise of AI has made good data engineering more valuable, not less. Feature pipelines, embeddings, and vector stores are increasingly core skills, and reliable, clean, well-governed data is what every model depends on.

*The clever model is only the last mile.*
*Someone must build the road that feeds it, and that is us.*

CHAPTER TWENTY-EIGHT

# Schema Evolution

*Schemas that grow up*

. . .

Arjun wanted to add one field to the orders, and he almost broke the whole company doing it.

It was a small, good idea. He wanted to add a tip field, so customers could tip their riders. One new field on the orders. Simple. He added it one afternoon, quickly, the way he'd built things in the early days, and by evening half the reports had stopped working and Kavya was staring at a wall of errors.

"He didn't do anything wrong, exactly," Meera told Mohan. "He added a field. That should be safe. But everything downstream, every report that reads orders, choked on it. A tiny change at the source, and the whole chain fell over." She frowned. "How do we ever change anything, once a hundred things depend on it?"

"Now you've reached one of the last hard things," Mohan said. "And it's the one that separates a system that lasts from one that slowly rots. Everything alive has to change. The question is how you change it without breaking everyone who was depending on the old shape." He pulled the laptop over. "Some changes are safe. Some are dangerous. And the difference is worth knowing in your bones."

"The shape of your data, the fields and their types, is called the schema," he said. "You've known that a long time. Changing it, as

the company grows, is called **schema evolution**. And there's a simple rule for which changes are safe." He drew two columns.

### SAFE VS DANGEROUS SCHEMA CHANGES

| change | safe? | why |
|---|---|---|
| add a new optional field | usually safe | old readers ignore it; missing = default |
| add a field with a default | safe | old rows get the default, not an error |
| rename a field | dangerous | every reader expecting the old name breaks |
| remove a field | dangerous | readers still using it break |
| change a field's type | dangerous | old data may no longer fit the new type |

*Adding is usually safe if old readers can ignore the new field. Renaming, removing, and retyping are the classic breakers, do them slowly, in stages.*

"Adding a new optional field is usually safe," he said. "Old readers just ignore what they don't know about. Adding a field with a sensible default is safe, old rows quietly get the default. But renaming a field is dangerous, because everything that looked for the old name suddenly can't find it. And removing a field is dangerous, because anything still using it breaks." He looked at her. "Add, gently: safe. Rename or remove: someone always falls."

"So Arjun's tip field should have been safe," Meera said. "He added a new field. Old readers should have just ignored it." She thought. "Unless he made it required. If a tip is required, every old order suddenly has a missing required field, and everything reading them panics."

Mohan pointed at her. "That's exactly what he did. He made it required. So make it optional, with a default of zero, and the whole problem vanishes. Old orders get a tip of zero. Nothing panics.

New orders can carry a real tip." He smiled. "The change was fine. The *way* he made it wasn't. Add gently, and the old world keeps working while the new one grows alongside it."

"Add gently. Never make the old world wrong just to let the new one in," Meera said. "You grow the shape without breaking what was already living in it."

"You grow the shape without breaking what was already there. That's the whole discipline." He looked at her a beat longer than the lesson needed. "It's a rare skill. Most people, to add something new, knock down what was there. The art is letting the new thing in *and* keeping the old thing whole."

· · ·

They fixed Arjun's tip field in ten minutes, once they knew the rule. And afterward, on the terrace, in the warm dark, Meera was quiet for a while, turning something over.

"You said something in there," she said. "About adding the new thing without breaking the old thing. Letting the new in and keeping the old whole." She looked at him. "That's what I've been afraid of, honestly. About us."

Mohan turned to her. "Tell me."

"For six years you were my best friend. The person I told everything. The friendship was the whole shape of us." She chose her words carefully, the way he'd taught her, long ago, to choose a schema change. "And now we've added this new thing. The love. And a small scared part of me worried that adding it would break the old thing. That I'd get the love and lose the friend. That the new field would knock down the old ones."

Mohan was quiet, listening, the way he did.

"But that's not what happened," Meera went on, working it out as she spoke, and smiling now. "We added gently. The friendship is all still there. Every bit of it. You're still the person I want to show things to first. We didn't rename it or remove it. We just... added a field. And the old rows are all still true." She laughed softly. "We evolved the schema safely. We got the new thing without breaking the old one."

Mohan laughed too, and pulled her closer under his arm.

"We evolved the schema safely," he agreed. "Best friends, still. Every column intact. Just one new field, added gently, with a good default." He kissed the top of her head. "You were always going to keep the friend, Meera. I'd have refused the love before I let it cost me that. Some fields you never remove, whatever else you add."

And the city hummed below them, and the old friendship and the new love sat side by side in the same true row, neither one breaking the other, the shape of them grown roomier without a single thing lost.

WHAT MEERA LEARNED

**Schema evolution** is the discipline of changing your data's shape over time without breaking everything that depends on it, because a living system's shape will keep changing for as long as it's used. The golden principle is **compatibility**: change things so the old and the new can coexist. **Backward compatibility** means new readers still handle old data (a missing field becomes a sensible default); **forward compatibility** means old readers still handle new data (they ignore fields they don't understand rather than crashing).

Adding a new optional, defaulted field is usually safe. Renaming, removing, or changing the type of a field are the classic breakers, do them in slow stages (add the new alongside the old, migrate gradually, remove the old only when nothing uses it). Never rip; always ease.

In a real job: schemas change constantly, and safe evolution (especially in streaming and shared data) is a genuinely senior skill. "Is this change backward compatible?" is a question worth asking before every schema edit.

*Your system is never finished, only alive.*
*So build a shape that can keep changing without breaking.*

CHAPTER TWENTY-NINE

# Cost & Judgement

*Wisdom and the cloud bill*

. . .

Nisha put a bill on the table one morning, and it was frightening.

The cloud bill. The cost of all those rented machines, all that stored data, all those questions asked and answered. In the early days it had been small enough to ignore. Now Zaayka was ten cities, and the bill had grown into a real number, the kind that makes a careful person like Nisha go quiet.

"We can afford it," Nisha said. "For now. But it's growing faster than we are. And I don't understand most of it. I just know we're paying for something, and I suspect a lot of it is waste." She looked at Meera. "Can you find out where the money's actually going?"

Meera took the bill to Mohan that evening, and he did not look worried. He looked, as ever, interested.

"A cloud bill frightens people because it looks mysterious," he said. "It isn't. You're really only ever paying for three things. Once you can see the three, you can find the waste in any bill." He drew them out.

### THE THREE TAPS, AND HOW TO CLOSE THEM

| you pay for | the waste | the fix you already know |
|---|---|---|
| **storage** | keeping everything forever | delete or archive what's truly unused |
| **compute** | jobs that run too often | run only as often as anyone needs |
| **scanning** | reading all data every time | partition & columnar so queries read less |

*Every fix is something Meera already learned, partitioning, columnar storage, sensible schedules, now pointed at the bill instead of at speed.*

"You pay to *keep* data, that's storage. You pay to *work* on it, that's the rented power, the compute. And you pay to *read* it, every time a question scans through it." He looked at her. "And here's the lovely part. Every fix for a high bill is something you already learned, chapters ago, for other reasons."

"Show me."

"Storage too high? You're keeping everything forever, including things nobody has touched in a year. So archive or delete what's truly unused, while keeping the raw you actually need. Compute too high? A job is running every hour when nobody needs it more than once a day. So run it only as often as someone actually reads it." He pointed at the third row. "And scanning too high? Your questions are reading all the data every time. But you already know the fix for that."

"Partition it," Meera said at once. "And store it by column. So a question about Hyderabad only reads Hyderabad, and a question about totals only reads totals. You stop reading what you don't need." She looked up. "These aren't new lessons at all. Partitioning, columns, keeping only the raw that matters. I learned all of these

for speed. They're the same fixes, just seen through money instead of time."

"The same fixes, seen through money instead of time," Mohan said. "That's the whole secret of cost work. A slow query and an expensive query are usually the same query. Fix it once, and you've paid for both." He smiled. "Making data work is a skill. Making it work *cheaply* is a more senior one. And it turns out to be mostly the tidiness you already believe in, counted in rupees."

· · ·

They went through the bill together, line by line, and found the waste, and closed the taps, and the next month's bill came in a third smaller, and Nisha nearly wept with relief and bought them both dinner.

But the thing Meera carried away from that month was a harder, quieter lesson, and it came near the end, when she wanted to build something clever to save even more.

"I could build a system," she said, "that automatically watches every job and tunes it and moves old data around, all on its own. It'd save us another slice. It'd be elegant. I kind of want to build it."

Mohan was quiet for a moment.

"How long to build it?" he asked. "And how much would it save, really, against the bill now that you've cleaned it?"

Meera did the sum honestly. "A month to build. And... not much. We already fixed the big leaks. It'd save a little, and cost me a month, and be one more clever thing that has to be maintained forever."

"So don't build it," Mohan said gently. "That's the last thing, Meera. And it's the hardest, because it goes against everything that makes you good at this. The most senior skill isn't building the

clever thing. It's knowing which clever thing *not* to build. Leaving the small leak alone, because chasing it costs more than it saves." He looked at her. "The wisdom isn't in what you make. Near the end, it's in what you have the restraint to leave unbuilt."

Meera sat with that, and found it landed somewhere deeper than the bill.

"That's not just about systems," she said slowly. "Is it."

"No," Mohan agreed. "It's about a life, too. Most of the peace I've ever found came from things I had the sense not to build. Worries I didn't construct. Cleverness I let go of." He took her hand. "Knowing when to stop building is most of wisdom. In data. And in everything else."

And they left the clever system unbuilt, and went home early instead, and Meera thought that of all the things she had learned in two years, this quiet one, the wisdom of the thing left alone, might be the one she'd needed most, and would carry longest, and had almost, in her cleverness, missed.

WHAT MEERA LEARNED

Making data work is a skill; making it work at the right cost is judgement, the discipline now called **FinOps**. Cloud money flows through three taps you already understand: **storage** (keeping everything forever), **compute** (jobs run too often), and **scanning** (reading all data every time). Every fix is a tool you already have, partitioning and columnar storage so queries read less, sensible schedules so jobs run only as needed, now pointed at the bill. Faster and cheaper are usually the same thing.

The most senior move of all is deciding *not* to build something. The dashboard nobody reads, the real-time feature nobody needs in real time, these are the most expensive things of all. The simplest system that solves the real problem is almost always the best; restraint is the last and hardest skill.

In a real job: cost awareness (FinOps) increasingly separates senior engineers from the rest. "Does this need to exist?" and "how much data does this scan?" are among the most valuable questions you can ask.

*The cheapest, fastest, wisest system*
*is often the one you had the restraint not to build.*

CLOSING

# Ten Cities Later

*Zaayka is in ten cities now. Meera holds the whole of it in her head, and understands, at last, what the job she stumbled into really was, and what, all along, had been quietly organising itself in her, waiting only to be asked.*

· · ·

CHAPTER THIRTY

# What a Data Engineer Really Is

*The question, asked again*

· · ·

Two years on, almost to the day, the same question came again. Only this time Meera was the one who knew the answer, and someone else was on the floor.

His name was Reza. He was new, and young, and earnest, and a little frightened, and he had joined Zaayka's data team three weeks ago. Zaayka was in ten cities now. Ten. Sri's shop was still kitchen 17, still the first row, still impossible to misspell, and Sri stuck his Friday numbers to the wall beside the specials and told every customer that a woman named Meera had given him a number that could never be forgotten.

And this morning Reza had come to Meera with a spreadsheet, and a question he could not answer, and the exact look she remembered wearing herself on a Sunday two years and a whole life ago.

· · ·

It was dawn. Meera came in early now; she liked the office before the others, when the light was still grey going gold and the whole system hummed quietly to itself, ten cities' worth of orders

flowing through pipes she had built, looking after themselves, whether or not anyone was watching.

Reza was already there, defeated by his spreadsheet, exactly as she had once been.

"I can't make it agree with itself," he said miserably. "The numbers won't line up. I've been here all night. I don't understand how anyone does this."

Meera did not laugh, though she wanted to, tenderly. She pulled up a chair. And she reached for the thing she had reached for a thousand times, the oldest tool she owned, older than the warehouse and the lake and the streams, the very first thing Mohan had ever given her.

"Let me tell you about a drawer," she said. "A long time ago, someone sat where you're sitting, and cried a little, if you must know. And a friend drew me a drawer full of little paper chits, all jumbled, and asked me a question I couldn't answer. And then he showed me that the problem was never that I lacked the data. I had all the data. The problem was that it had no shape." She turned Reza's screen toward her. "Having data and being able to use it are two different things. The work is in the organising. Done before the asking. Everything else you'll ever learn here is just that, again and again, at bigger and bigger sizes. Now. Show me your drawer."

And she taught him. The way she had been taught. Patiently, warmly, one shape at a time, in the grey-gold light of the dawn, while the tulsi on the office windowsill kept its small green watch, the same tulsi she had carried from flat to flat to flat, older now, and thriving.

· · ·

Mohan found them there an hour later, heads bent over the spreadsheet, Reza beginning, slowly, to see.

He did not interrupt. He stood in the doorway with two cups of chai, one of which he would drink and one of which he still, after everything, refused to. He watched the woman he had taught teach someone else, using his own oldest words. And something turned over in his chest, the way it had on the very first evening. Except that now he let it show.

Because he could see the whole shape of it from the doorway. The drawer he had drawn for her, she was drawing for Reza. The lesson travelling on. And under the lesson, quieter, the other thing travelling on with it: the patient, invisible organising of one life around another, done long before either thought to ask.

When Reza finally left, clutching his first honest table like a lamp, Mohan crossed the room and set the chai beside her.

"You've become the drawer," he said.

"I've become the drawer." She wrapped her hands around the warm cup. "Can I tell you something I worked out? At dawn, while I was teaching him?"

"Always."

· · ·

Outside, over the waking city, a temple bell rang. Three unhurried notes, the way it had rung on the very first evening, two years ago, when she had barely known what a database was. Three notes, patient, spaced, complete. Meera waited for them to finish before she spoke, the way you wait for something you love to finish speaking.

"You told me once," she said, "that there were three ways to learn anything that matters. And you gave me two, and said I'd

taught you the third. I've been thinking about them for a year. And this morning, teaching Reza, I finally understood how they fit." She looked at him, and the dawn was full gold now on her face. "There's seeing a thing truly. The actual shape, under all the mess. That's the first. And there's doing the work, the patient organising, for its own sake, without grasping at the reward. That's the second." She paused. "And there's loving a thing enough to get it right. Not for the reward. Not even to understand it. Just because you love it, and love won't let you leave it half-true."

"Seeing, working, loving," Mohan said quietly.

"Thinking, working, loving. And here's the thing I finally saw this morning." Her voice was very steady, and very sure, the voice of a woman who had crossed the whole distance from that first Sunday floor to here. "They're not three separate ways. They're one way, met three times. When you truly see a thing, you start to care for it. When you care for it, you do the patient work. And when you do the work, without grasping, only because you love the thing enough to get it right, you begin, finally, to see it truly. Round and round. Seeing becomes loving becomes working becomes seeing." She smiled. "That's the whole of it. That's the whole of everything. The organising, done before the asking, was never only about data. It was how to see. It was how to work. It was how to love. It was the same lesson, all along, wearing thirty different coats."

Mohan did not say anything for a long moment. The bell had faded. The city hummed. The tulsi kept its watch.

"I spent fifteen years learning that," he said at last, "and I never once said it as well as you just did. At dawn. Off the top of your head. To explain why you're happy." He shook his head slowly, wonderingly. "You were never my student. I see that now. You were the thing I was organising my whole life toward, long before I

knew to ask. And when the question finally came, the answer was just there. Ready. True."

· · ·

They stood at the window together and watched the gold come up over the ten cities they could not see but knew were there. Waking. Ordering breakfast. Each order flowing safely home through the honest pipes of a thing a frightened woman had built, with a good pen, and a patient friend, and a great deal of love she had not, for the longest time, known how to name.

Somewhere out there, Suresh was starting his scooter. Somewhere, Priya was ordering her dosa, safe behind a guarded door. Somewhere, Lakshmi was counting her biryani by hand and trusting a number she had never touched. Somewhere, Sri's Friday total was going up on the wall. Ten cities of small kindnesses, flowing, live, each one a person carrying someone's warm breakfast through the gold morning, each with someone waiting at a window.

And Meera thought about the watermark. The line that waits, patiently, on purpose, for the ones still in the tunnel. That holds the door open exactly long enough. That trusts late is not the same as lost.

She had waited. She had held her door. And the thing that was only late, not lost, had come through it at last, on a Thursday, and stayed.

"Watermark reached," she said softly. To the gold morning. To the ten cities. To the man beside her. To the whole patient architecture of a life that had built itself in careful invisible layers, while they both, for years, looked the other way. "No more stragglers. Everything's arrived."

Mohan took her hand on the windowsill, the way a customer and a kitchen finally sit in the same honest row, and together they watched the dawn finish arriving over everything they had made, and everything they had, without ever once rushing it, quietly become.

### THE WHOLE BOOK, IN ONE BREATH

Everything in these thirty chapters grew from a single idea Meera stumbled onto on her first Sunday: *having data and being able to use it are two different things, and the work is in the organising, done before the asking.*

Tables and keys, types and transactions, warehouses and lakes, streams and pipelines, models and governance, cost and restraint, none of it was a separate lesson. It was that one idea, met again and again at larger and larger scale. A data engineer is someone who makes data trustworthy: who gives the mess of the world a shape, so that people can ask honest questions and get honest answers, and never have to think about how.

*The work is in the organising.*
*Do it well, and then go teach someone the drawer.*

APPENDIX A

# The Canonical Dataset

*Every table the story draws from, in full. Every number in the novel is a true slice of what follows. Money is stored exactly, in paise, and shown here in rupees.*

· · ·

APPENDIX A

# The Data in Full

These are the tables behind Zaayka, exactly as the book uses them. Every kitchen is pure vegetarian, as Zaayka always was.

### CITIES

| city_id | name | launched |
|---|---|---|
| 1 | Bengaluru | 2026-01-05 |
| 2 | Hyderabad | 2026-05-01 |
| 3 | Pune | 2026-08-01 |

### AREAS

| area_id | city_id | name |
|---|---|---|
| 1 | 1 | Malleshwaram |
| 2 | 1 | Rajajinagar |
| 3 | 1 | Indiranagar |
| 4 | 2 | Banjara Hills |
| 5 | 2 | Gachibowli |
| 6 | 3 | Koregaon Park |

**KITCHENS**

| kitchen_id | name | area_id | area | since |
|---|---|---|---|---|
| 17 | Sri Idli | 1 | Malleshwaram | 2026-01-05 |
| 18 | Krishna Bakery | 1 | Malleshwaram | 2026-01-05 |
| 19 | Lakshmi's Biryani | 2 | Rajajinagar | 2026-01-12 |
| 20 | Anand Tiffins | 3 | Indiranagar | 2026-02-02 |
| 21 | Chai Point Corner | 1 | Malleshwaram | 2026-02-14 |
| 22 | Paradise Dosa | 2 | Rajajinagar | 2026-03-01 |
| 23 | Nandini Sweets | 3 | Indiranagar | 2026-03-14 |

*Nandini Sweets (23) has a menu but no orders, the standing example for outer joins.*

### DISHES

| dish_id | kitchen_id | kitchen | name | price |
|---|---|---|---|---|
| 101 | 17 | Sri Idli | Idli plate | ₹90 |
| 102 | 17 | Sri Idli | Masala dosa | ₹120 |
| 103 | 17 | Sri Idli | Ghee pongal | ₹110 |
| 104 | 17 | Sri Idli | Medu vada | ₹80 |
| 105 | 17 | Sri Idli | Filter coffee | ₹40 |
| 106 | 17 | Sri Idli | Rava dosa | ₹120 |
| 201 | 18 | Krishna Bakery | Veg puff | ₹30 |
| 202 | 18 | Krishna Bakery | Bun | ₹20 |
| 203 | 18 | Krishna Bakery | Dilkhush | ₹50 |
| 301 | 19 | Lakshmi's Biryani | Paneer biryani | ₹220 |
| 302 | 19 | Lakshmi's Biryani | Mushroom biryani | ₹280 |
| 303 | 19 | Lakshmi's Biryani | Veg biryani | ₹180 |
| 304 | 19 | Lakshmi's Biryani | Raita | ₹30 |
| 401 | 20 | Anand Tiffins | Set dosa | ₹100 |
| 402 | 20 | Anand Tiffins | Kesari bath | ₹60 |
| 501 | 21 | Chai Point Corner | Cutting chai | ₹15 |
| 502 | 21 | Chai Point Corner | Samosa | ₹25 |
| 601 | 22 | Paradise Dosa | Paper dosa | ₹110 |
| 602 | 22 | Paradise Dosa | Onion uttapam | ₹100 |
| 701 | 23 | Nandini Sweets | Mysore pak | ₹60 |
| 702 | 23 | Nandini Sweets | Jalebi | ₹40 |

**CUSTOMERS**

| customer_id | name | area_id | joined |
|---|---|---|---|
| 318 | Priya | 1 | 2026-01-08 |
| 206 | Rahul | 1 | 2026-01-10 |
| 442 | Anjali | 2 | 2026-01-20 |
| 517 | Vikram | 3 | 2026-02-05 |
| 623 | Fatima | 1 | 2026-02-11 |
| 704 | Deepak | 2 | 2026-03-01 |

**RIDERS**

| rider_id | name | city_id |
|---|---|---|
| 51 | Suresh | 1 |
| 52 | Manoj | 1 |
| 53 | Ganesh | 1 |

### ORDERS

| order_id | customer_id | kitchen_id | placed_at | status | delivered_at |
|---|---|---|---|---|---|
| 4402 | 318 | 17 | 2026-03-10 20:04 | delivered | 2026-03-10 20:38 |
| 4404 | 318 | 17 | 2026-03-10 21:55 | cooking | NULL |
| 4419 | 206 | 17 | 2026-03-10 13:12 | delivered | 2026-03-10 13:44 |
| 4431 | 623 | 17 | 2026-03-10 21:37 | delivered | 2026-03-10 22:09 |
| 4433 | 442 | 18 | 2026-03-10 18:20 | delivered | 2026-03-10 18:41 |
| 4440 | 704 | 18 | 2026-03-10 19:05 | delivered | 2026-03-10 19:33 |
| 4451 | 517 | 19 | 2026-03-10 20:50 | delivered | 2026-03-10 21:35 |
| 4460 | 206 | 19 | 2026-03-11 12:30 | cancelled | NULL |
| 4472 | 442 | 22 | 2026-03-11 09:15 | delivered | 2026-03-11 09:47 |
| 4488 | 517 | 20 | 2026-03-11 08:40 | delivered | 2026-03-11 09:10 |

*Totals are computed from the items below, never typed. Order 4404 is still cooking (NULL delivery); 4460 was cancelled.*

### ORDER_ITEMS

| item_id | order_id | dish | qty | unit_price | line_total |
|---|---|---|---|---|---|
| 9001 | 4402 | Masala dosa | 2 | ₹120 | ₹240 |
| 9002 | 4402 | Idli plate | 1 | ₹90 | ₹90 |
| 9003 | 4402 | Filter coffee | 1 | ₹40 | ₹40 |
| 9004 | 4419 | Ghee pongal | 1 | ₹110 | ₹110 |
| 9005 | 4419 | Filter coffee | 1 | ₹40 | ₹40 |
| 9006 | 4431 | Idli plate | 1 | ₹90 | ₹90 |
| 9007 | 4431 | Medu vada | 1 | ₹80 | ₹80 |
| 9008 | 4433 | Veg puff | 4 | ₹30 | ₹120 |
| 9009 | 4433 | Dilkhush | 2 | ₹50 | ₹100 |
| 9010 | 4440 | Bun | 3 | ₹20 | ₹60 |
| 9011 | 4440 | Veg puff | 2 | ₹30 | ₹60 |
| 9012 | 4451 | Paneer biryani | 2 | ₹220 | ₹440 |
| 9013 | 4451 | Raita | 1 | ₹30 | ₹30 |
| 9014 | 4404 | Masala dosa | 1 | ₹120 | ₹120 |
| 9015 | 4404 | Idli plate | 1 | ₹90 | ₹90 |
| 9016 | 4460 | Veg biryani | 1 | ₹180 | ₹180 |
| 9017 | 4472 | Paper dosa | 1 | ₹110 | ₹110 |
| 9018 | 4472 | Onion uttapam | 1 | ₹100 | ₹100 |
| 9019 | 4488 | Set dosa | 2 | ₹100 | ₹200 |
| 9020 | 4488 | Kesari bath | 1 | ₹60 | ₹60 |

**FAVOURITES** · many-to-many join table

| customer_id | customer | kitchen_id | kitchen |
|---|---|---|---|
| 318 | Priya | 17 | Sri Idli |
| 318 | Priya | 19 | Lakshmi's Biryani |
| 206 | Rahul | 17 | Sri Idli |
| 442 | Anjali | 19 | Lakshmi's Biryani |
| 623 | Fatima | 18 | Krishna Bakery |

APPENDIX B

# The Map

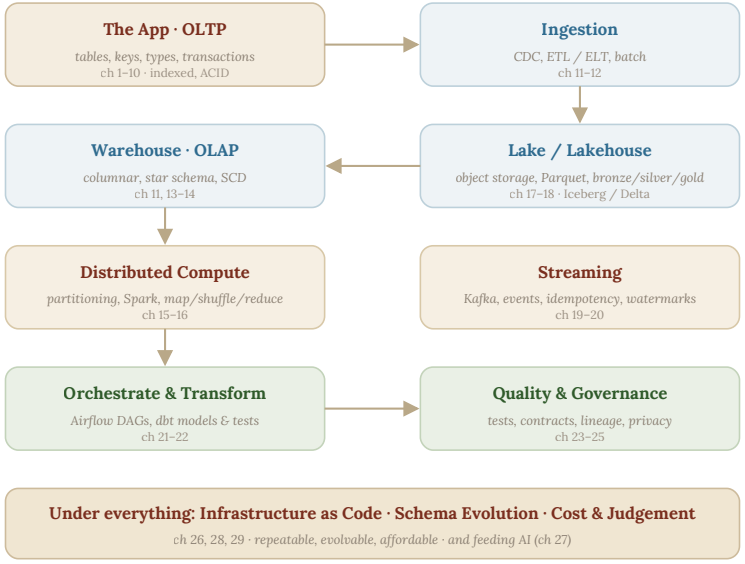
*The whole modern data stack on one page, in the order Meera met it. Every box is a chapter. Follow an order from the app, on the left, all the way to a decision, on the right.*

APPENDIX B

# The Whole System, at a Glance

. . .

Here is the journey of a single order, from the moment a customer taps their screen to the moment a number helps someone decide something. Every stage is a chapter you have read.




*The whole modern data stack. An order flows from the app through ingestion into the warehouse and lake, is processed (in batch or as a stream), transformed and orchestrated, checked and governed, all resting on a foundation of code, safe change, and cost sense.*

Notice that the deepest ideas repeat at every layer. *Columnar storage* appears in the warehouse and again in Parquet files. *Partitioning* speeds queries and saves cost. *Idempotency* guards streams and pipelines alike. *Write it down, make it repeatable* begins with a single order's shape and ends with a whole company's infrastructure. You did not learn thirty separate things. You learned a handful of ideas, each meeting you again, larger, until they became a system.

APPENDIX C

# SQL Quick Reference

*Every query pattern the book taught, gathered in one place, so this book can be something you return to. All examples run against the canonical Zaayka data.*

APPENDIX C

# The SQL You Learned

. . .

These are the shapes of question you now know. Keep them close; they answer most of what a business ever asks.

**Show rows, choose columns, filter.** The everyday trio: pick columns, name the table, keep only rows that pass a test.

```sql
-- Sri Idli's orders on one Tuesday
SELECT order_id, placed_at, total
FROM orders
WHERE kitchen_id = 17
  AND placed_at >= '2026-03-10'
  AND placed_at < '2026-03-11'
ORDER BY total DESC;
```

**Filter to a single day the safe way:** a range from the day up to (but not including) the next day, never = a date, because a day is a span of moments, not one instant.

**Group and measure.** Gather rows into piles by some column, then measure each pile with an aggregate: COUNT, SUM, AVG, MIN, MAX.

```sql
-- orders and revenue per kitchen
SELECT kitchen_id,
       COUNT(*)     AS order_count,
       SUM(total)   AS revenue,
       AVG(total)   AS avg_order
FROM orders
WHERE status = 'delivered'
GROUP BY kitchen_id;
```

**Join tables back together.** Bring facts from two tables into one answer by matching on a shared key. Give tables short aliases for clarity.

```sql
-- revenue per kitchen, by name
SELECT k.name,
       COUNT(*)     AS order_count,
       SUM(o.total) AS revenue
FROM orders o
JOIN kitchens k ON k.kitchen_id = o.kitchen_id
WHERE o.status = 'delivered'
GROUP BY k.name;
```

**Keep the empties with an outer join.** A plain (inner) JOIN drops rows with no match, so a kitchen with zero orders vanishes. A LEFT JOIN keeps every kitchen and shows zero for the missing side.

```sql
-- every kitchen, even ones with no orders (e.g. Nandini Sweets, 23)
SELECT k.name, COUNT(o.order_id) AS order_count
FROM kitchens k
LEFT JOIN orders o ON o.kitchen_id = k.kitchen_id
GROUP BY k.name;
```

**Do several steps safely, all or nothing.** Wrap steps that must not be half-done, or must not tangle with others, in a transaction.

```sql
BEGIN;
  UPDATE stock
  SET qty = qty - 1
  WHERE dish_id = 302 AND qty >= 1;
COMMIT;
```

**Make a lookup fast.** Add an index on a column you often search by. It speeds reads, at a small cost to writes and space.

```sql
CREATE INDEX idx_orders_customer ON orders (customer_id);
```

### THE WORDS, AT A GLANCE

| word | does |
|---|---|
| SELECT | choose which columns to show (* = all) |
| FROM | name the table |
| WHERE | keep only rows passing a test (stack with AND/OR) |
| ORDER BY | sort the result (DESC = largest first) |
| GROUP BY | gather rows into piles by a column |
| COUNT/SUM/AVG | measure each pile |
| JOIN ... ON | match two tables on a shared key |
| LEFT JOIN | as above, but keep unmatched rows too |
| BEGIN/COMMIT | run steps as one all-or-nothing transaction |
| CREATE INDEX | add a sorted lookup to speed searches |

APPENDIX D

# "Your Turn" Answers

*Worked solutions to the optional reader prompts scattered through the book. If you tried them, well done; that trying is where learning actually happens.*

· · ·

APPENDIX D

# Worked Answers

. . .

Here are the answers to the "Your turn" questions, with the reasoning, not just the result, because the reasoning is the point.

**Chapter 8 · the two paneer biryanis.** Lakshmi has exactly two paneer biryanis left, and three customers order at the same instant. Each order runs, inside its own transaction, the safe step: *take one only if at least one is left* (UPDATE … SET QTY = QTY - 1 WHERE QTY >= 1).

Because each runs in a transaction, the database lines them up and lets only one act at a time, no two can slip into the gap between checking and taking. So they happen in some order, one after another:

THREE ORDERS, TWO BIRYANIS, ONE AT A TIME

| order | stock before | qty >= 1? | result | stock after |
|---|---|---|---|---|
| first | 2 | yes | takes one | 1 |
| second | 1 | yes | takes one | 0 |
| third | 0 | no | kindly turned away | 0 |

So **two orders succeed and one is turned away**, and the stock ends at exactly zero. No biryani is sold twice, and none is wrongly refused while stock remained. That is exactly what the transaction

plus the QTY >= 1 guard buy you: correctness even when everything happens at once.

The deeper lesson: you did not need to control *when* the three orders arrived. You only needed to make each one's check-and-take indivisible. The messiness of timing became harmless, which is the whole art of handling things that happen at once.

. . .

*More "Your turn" prompts appear as the book grows.*
*The habit of trying them first is worth more than any single answer.*